

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPD801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

27th May 2026



Abstract

Words words words words words words words words words
Ord ord ord ord ord ord ord ord ord ord

CONTENTS

1	Introduction	3
2	Background	4
2.1	Classical Linear Logic, CLL	4
2.2	The π -Calculus: Processes and Communication	5
2.3	Types and Propositions-as-Sessions Correspondence	6
2.4	Environments, Hyperenvironments and Judgements	7
2.5	Operational Semantics	8
2.6	Process Semantics and Erasure	10
2.7	Semantics of Typing Environments	12
2.8	Managing Bound Variables	14
3	Binding Architecture	15
3.1	Design Goals and the Approach	15
3.2	The Locally Nameless Representation	15
3.3	Opening and Closing Binders	16
3.4	Co-finite Quantification	17
4	Formalisation	19
4.1	Model	19
4.2	Semantics	29
5	Metatheory	31
5.1	Session Fidelity	31
6	Discussion	33
7	Future Work	35
8	Conclusion	37
	References	38
A	Code Availability	38
B	Lean File Structure	39
C	Common Lean Notation	39
D	Selected Core Definitions	40
D.1	Session Types	40
D.2	Processes	43
D.3	Environments	48
D.4	Hyperenvironments	51
E	Selected Transition Definitions	55
F	Erasure Functions and Main Theorems	56
G	Selected Proof Infrastructure	56

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π -calculus.

This line of work has been further developed by [Montesi and Peressotti \[2021\]](#), who provide a unified account reconciling linear logic, the π -calculus, and their metatheory. Their framework serves as the formal basis for the system π LL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, a small mistake in the application of a typing rule can arise when the intuitive order of names does not match the formal order imposed by the rule. It is tempting to assign types following the order in which names appear syntactically, but a rule may assign types according to the role each name plays instead. Thus, despite the rule itself being correct, its application is easy to get wrong because the intuitive and formal interpretations pull in opposite directions. Such mistakes do not reflect a flaw in the underlying theory, but they are easy to overlook in paper presentations where rule application is guided by intuition rather than by strict syntactic structure. This makes the system a natural candidate for mechanisation, since in proof assistants such as Lean, every rule application must instantiate the required variables and side conditions explicitly.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from paper presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π -calculus-based linear logic system π LL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalisation of the syntax and typing system of π LL in Lean, including processes, session types, environments, hyperenvironments, typing judgements, and substitution operations for both names and types.
- A binding architecture based on De Bruijn indices for type variables and a locally nameless representation for process binders, avoiding explicit reasoning up to α -equivalence.
- A formalisation of the operational semantics for the multiplicative fragment of π LL.

- A mechanised development of the session-fidelity proof architecture for the multiplicative fragment, with the remaining proof obligation isolated to the `res` case.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the π LL system of [Montesi and Peressotti 2021], with particular emphasis on its multiplicative fragment. The syntactic and typing components of π LL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, while the corresponding session-fidelity proof currently leaves the `res` case as the remaining proof obligation.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanised account of an existing theoretical framework and to identify the remaining proof obligations needed for its full verification.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021]. We cover the full π LL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment π MLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \wp B$, together with the corresponding units 1 and $\perp$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \wp B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B , whereas $A \wp B$ corresponds to receiving a value of type A and proceeding as B . The units 1 and $\perp$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1} \quad \frac{\vdash \Gamma}{\vdash \Gamma, \perp} \perp \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B} \otimes \quad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \wp B} \wp$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B , each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B .

The unit rule [rule 1](#) introduces the unit 1 , which represents the absence of further obligations. It can be understood as an “empty output”, corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, [rules \$\perp\$](#) and [\$\wp\$](#) describe the behavior of the connective $A \wp B$ and its unit $\perp$. The [\$\wp\$](#) rule combines resources within a single context, reflecting a form of interaction where both components are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B . The unit $\perp$ represents the absence of further input, acting as an “empty input” and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the [rule 1](#) and combined via [rule \$\otimes\$](#) to derive $1 \otimes 1$:

$$\frac{\frac{}{\vdash 1} 1 \quad \frac{}{\vdash 1} 1}{\vdash 1 \otimes 1} \otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying π LL.

2.2 The π -Calculus: Processes and Communication

The π -calculus [[Milner et al. 1992](#)] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of [Montesi and Peressotti \[2021\]](#), which adopts [Wadler’s](#) convention of using square brackets ‘[-]’ for output and round parentheses ‘(-)’ for input.

Programs in π MLL are processes $(P, Q, R, \dots)$, which communicate over names $(x, y, z, \dots)$ representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in [2.3](#). We assume an infinite set of names with decidable equality. The process terms of π MLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	<i>output y on x and continue as P</i>
	$x(y).P$	<i>input y on x and continue as P</i>
	$x[].P$	<i>output (empty message) on x and continue as P</i>
	$x().P$	<i>input (empty message) on x and continue as P</i>
	$\nu xy.P$	<i>name restriction (cut)</i>
	$P \mid Q$	<i>parallel composition</i>
	$x[L].P$	<i>select left on x and continue as P</i>
	$x[R].P$	<i>select right on x and continue as P</i>
	$x.\text{case}\{L:P, R:Q\}$	<i>offer a binary choice between P or Q on x</i>
	$x[A].P$	<i>output type A on x and continue as P</i>
	$x(X).P$	<i>input a type X on x and continue as P</i>
	$!x\langle z_1, \dots, z_n \rangle. \{P\}$	<i>server (replicable process) on x depending on servers on $z_1 \dots z_n$</i>
	$x[\text{USE}].P$	<i>consume a server on x and continue as P</i>
	$x[\text{DUP}](x').P$	<i>duplicate server x as x' in P</i>
	$x[\text{DISP}].P$	<i>dispose of a server on x</i>

	$x \leftrightarrow y$	link x and y
	0	terminated process

Term $x[y].P$ allocates a fresh name y , outputs it over x , and proceeds as P . Dually, $x(y).P$ inputs a name over x , binding it to y in P . Terms $x[].P$ and $x().P$ model empty output and input. Restriction $\nu xy.P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\nu xy.P = \nu yx.P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and 0 is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \nu xz (x[].0 \mid z().y[].0)$. This process creates a private session between the restricted endpoints x and z . The left process, $x[].0$, signals termination on x before terminating, while the right process, $z().y[].0$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for π LL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In π LL, session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B , while its dual $A \wp B$ describes receiving. The units 1 and $\perp$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \& B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of π LL is defined as follows:

$A, B ::=$	$A \otimes B$	send A , proceed as B		$A \wp B$	receive A , proceed as B
	1	empty output, unit for $\otimes$		$\perp$	empty input, unit for $\wp$
	$A \& B$	offer A or B		$A \oplus B$	select A or B
	$\exists X.A$	existential, type output		$\forall X.A$	universal, type input
	$?A$	client request		$!A$	server accept
	X	type variable		$^\perp X$	dual of type variable

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, i.e. $(A^\perp)^\perp = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$\begin{aligned}
 (A \otimes B)^\perp &= A^\perp \wp B^\perp & 1^\perp &= \perp & (A \oplus B)^\perp &= A^\perp \& B^\perp \\
 (!A)^\perp &= ?A^\perp & (\exists X.A)^\perp &= \forall X.A^\perp & (X)^\perp &= X^\perp
 \end{aligned}$$

2.4 Environments, Hyperenvironments and Judgements

Typing in π LL is organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \dots, x_n : A_n$, where each x_i is associated to its respective A_i , for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using “,” as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \quad \Gamma, \Delta = \Delta, \Gamma \quad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator ‘|’. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} | \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using ‘|’ the sum and $\emptyset$ as unit, where $\emptyset$ is the hyperenvironment containing no environments:

$$\mathcal{G} | \emptyset = \mathcal{G} \quad \mathcal{G} | \mathcal{H} = \mathcal{H} | \mathcal{G} \quad (\mathcal{G} | \mathcal{H}) | \mathcal{I} = \mathcal{G} | (\mathcal{H} | \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\vdash P :: \mathcal{G}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P , appearing in the judgement concluding a derivation is well-typed.

Multiplicative Fragment

$$\begin{array}{c} \frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^\perp}{\vdash vxy.P :: \mathcal{G} \mid \Gamma, \Delta} \text{CUT} \quad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \text{MIX} \quad \frac{}{\vdash 0 :: \emptyset} \text{MIX}_0 \\ \frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B} \otimes \quad \frac{\vdash P :: \emptyset}{\vdash x[] . P :: x : 1} 1 \quad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \wp B} \wp \quad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \perp} \perp \end{array}$$

Additional rules for Additives, Quantifiers and Exponentials

$$\begin{array}{c} \frac{\vdash P :: \Gamma, x : A}{\vdash x[L].P :: \Gamma, x : A \oplus B} \oplus_1 \quad \frac{\vdash P :: \Gamma, x : B}{\vdash x[R].P :: \Gamma, x : A \oplus B} \oplus_2 \quad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{L:P, R:Q\} :: \Gamma, x : A \& B} \& \\ \frac{}{\vdash x \leftrightarrow y :: x : A^\perp, y : A} \text{AX} \quad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A} ? \quad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A} \text{W} \quad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A} \text{C} \\ \frac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \dots, z_n \rangle . \{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A} ! \quad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B} \exists \quad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B} \forall \end{array}$$

Fig. 1. π LL, typing rules.

The multiplicative rules form the core of the system. **Rule** **CUT** composes two processes in P sharing dual endpoints $x : A$ and $y : A^\perp$, hiding them via restriction. This is the logical equivalent of communication. **Rule** **MIX** and **rule** **MIX₀** compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and **rule** $\wp$ type output and input. Note that **rule** $\otimes$ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, **rule** $\wp$ keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units **rule** **1** and **rule** $\perp$ type empty output and empty input, where **rule** **1** requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to 0 .

The additive rules (**rule** $\oplus_1$, **rule** $\oplus_2$, and **rule** $\&$) type selection and offering of alternative processing paths. Note that **rule** $\&$ requires both branches to be typed in the *same* context Γ , reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The **rule** **!** requires all free names except x to be typed using $?$, enforcing that a server only depends on other servers. The **rule** $?$, **rule** **w**, and **rule** **c** allow a client to use, discard, or duplicate a server respectively.

The **rule** **ax** types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. **Rule** $\exists$ and **rule** $\forall$ type polymorphic communication, where the side condition $X \notin \text{ft}(\Gamma)$ in **rule** $\forall$, ensures the introduced type variable, X , does not accidentally shadow one already existing in Γ .

Example 2.2. The process from **Example 2.1** has the typing $\vdash \mathbf{v}zx(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : \mathbf{1}$, as shown by the derivation below:

$$\begin{array}{c}
\frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \quad \frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash y[].\mathbf{0} :: y : \mathbf{1}} \mathbf{1}}{\vdash z().y[].\mathbf{0} :: y : \mathbf{1}, z : \perp} \perp}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : \mathbf{1} \mid y : \mathbf{1}, z : \perp} \text{MIX}}{\vdash \mathbf{v}zx(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : \mathbf{1}} \text{CUT}
\end{array}$$

Note that, for the typing context $x : \mathbf{1} \mid y : \mathbf{1}, z : \perp$ to align with the premise of **rule** **CUT**, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \emptyset$ and $\Gamma = \bullet$. Under these instantiations, the premise of **rule** **CUT**: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^\perp$ can be viewed as $x : A \mid \Delta, y : A^\perp$. This effectively instantiates the **CUT** over the dual types $A = \mathbf{1}$ and $A^\perp = \perp$ on channels x and y , with $\Gamma = y : \mathbf{1}$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid **CUT** (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in **Figure 2**. The full LTS is given in **Montesi and Peressotti [2021]**.

Remark 2.3. The operational semantics rely explicitly on the structural equivalence (**rule** $=_\alpha$) to rename restricted variables via α -equivalence prior to a transition. In paper-based presentations, α -equivalence is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting, however, reasoning over named variables requires freshness conditions, capture-avoiding

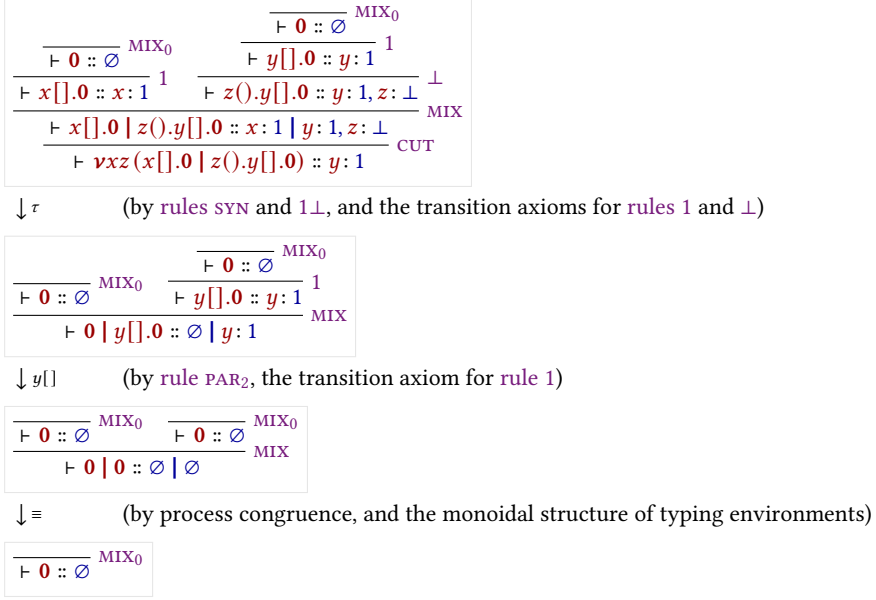


Fig. 3. An execution for the derivation in Example 2.2.

substitution, and explicit renaming lemmas, introducing substantial proof overhead [Aydemir et al. 2005; Charguéraud 2012]. As discussed in Section 3, this motivates the use of a different binding representation for the syntax of π LL, one designed to reduce explicit α -equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l \mid l') \mid l, l' \in Act, i(l) \cap i(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

Example 2.4. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1 , $\perp$, $\otimes$, $\wp$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule **syn** allows for the pairing of concurrent actions via $l \mid l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\perp$ and $\otimes\wp$ simplify the term into an internal τ -transition, modeling the synchronisation of the session.

2.6 Process Semantics and Erasure

While the typing derivations of π LL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π -calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \rightarrow Proc$, which strips away the typing

information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed π LL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\begin{array}{c}
 \frac{}{\vdash 0 :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash 0 :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash 0 :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash y[] . 0 :: y : 1} 1 \\
 \frac{}{\vdash v[] . 0 :: v : 1} 1 \quad \frac{}{\vdash x[] . 0 :: x : 1} 1 \quad \frac{}{\vdash z() . y[] . 0 :: y : 1, z : \perp} \perp \\
 \frac{}{\vdash w() . v[] . 0 :: v : 1, w : \perp} \perp \quad \frac{}{\vdash x[] . 0 \mid z() . y[] . 0 :: x : 1 \mid y : 1, z : \perp} \text{MIX} \\
 \frac{}{\vdash (w() . v[] . 0 \mid x[] . 0 \mid z() . y[] . 0) :: v : 1, w : \perp \mid x : 1 \mid y : 1, z : \perp} \text{MIX} \\
 \frac{}{\vdash vxz(w() . v[] . 0 \mid x[] . 0 \mid z() . y[] . 0) :: v : 1, w : \perp \mid y : 1} \text{CUT}
 \end{array}$$

Fig. 4. A derivation for the process $vxz(w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0)$

Example 2.5. Consider the well-typed process P formed by adding a third parallel component to the derivation of [Example 2.2](#) before the cut (A derivation of which can be seen on [Figure 4](#)):

$$P = vxz(w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0)$$

The component containing w and v is independent from the component containing x , z , and y . This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$\begin{array}{lll}
 P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{y[]} 0 & P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{y[]} \xrightarrow{v[]} 0 & P \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{w()} \xrightarrow{v[]} 0 \\
 P \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{\tau} \xrightarrow{y[]} 0 & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{v[]} \xrightarrow{y[]} 0 & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{v[]} 0
 \end{array}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in [[Montesi and Peressotti 2021](#)], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times \text{Lbl}$ be endofunctors representing the states and labeled transitions, respectively. The erasure function $proc$ acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

THEOREM 2.6 (ERASURE). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in \text{Lbl}$:*

- (1) **Soundness of Erasure:** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*
- (2) **Completeness of Erasure:** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

COROLLARY 2.7 (TYPABILITY PRESERVATION). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for π LL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

$$\begin{array}{c}
\begin{array}{c}
x[x'].P \xrightarrow{x[x']} P \quad x(x').P \xrightarrow{x(x')} P \quad x[].P \xrightarrow{x[]} P \quad x().P \xrightarrow{x()} P \\
\frac{P \xrightarrow{l} P' \quad i(l) \cap f(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q} \text{PAR}_1 \quad \frac{Q \xrightarrow{l} Q' \quad i(l) \cap f(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'} \text{PAR}_2 \\
\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad i(l \parallel l') \cap f(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l \parallel l'} P' \mid Q'} \text{SYN} \quad \frac{P \xrightarrow{x[x'] \parallel y(y')} P'}{vxy P \xrightarrow{\tau} vxy vx'y' P'} \otimes \otimes \\
\frac{P \xrightarrow{x[] \parallel y()} P'}{vxy P \xrightarrow{\tau} P'} 1\bot \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{vxy P \xrightarrow{l} vxy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha}
\end{array}
\end{array}$$

$$\begin{array}{c}
i(l \parallel l') = i(l) \cup i(l') \\
f(l \parallel l') = f(l) \cup f(l') \\
i(\tau) = \emptyset \\
f(\tau) = \emptyset \\
i(x[]) = i(x()) = \emptyset \\
f(x[]) = f(x()) = \{x\} \\
i(x[x']) = i(x(x')) = \{x'\} \\
f(x[x']) = f(x(x')) = \{x\}
\end{array}$$

Fig. 5. π MLL, transition rules for processes.

Remark 2.8. The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because π LL enforces linear resource management, the structural rules of the typed LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

PROPOSITION 2.9. *The operational semantics of π LL processes strictly respects the introduction of names via transition labels. Formally, if $P \xrightarrow{l} P'$, then $i(l) \cap f(P) = \emptyset$, and If P is well-typed, then $i(l) = f(P') \setminus f(P)$ (TODO: Evaluate relevance - delete depending on if this property is mentioend in later proofs).*

Remark 2.10. **Proposition 2.9** is vital for mechanising the metatheory of π LL. Because proof assistants tools require explicit tracking of fresh and bound variable to prevent accidental capture, this proposition acts as the formal bridge between the dynamic semantics and the static typing. It guarantees that a well-typed process evolves predictably, ensuring that it does not spontaneously generate resources during a transition. (TODO: same as **Proposition 2.9**)

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In π LL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow **Montesi and Peressotti [2021]** and define a function $env : Der \rightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, **Montesi and Peressotti [2021]** derive a LTS specifically for typing environments (lts_e), shown in **Figure 6**.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \perp$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalised by the τ -reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ -action.

$$\begin{array}{c}
x:1 \xrightarrow{x[]} \emptyset \quad \Gamma, x:\perp \xrightarrow{x()} \Gamma \quad \Gamma, \Delta, x:A \otimes B \xrightarrow{x[x']} \Gamma, x':A \mid \Delta, x:B \quad \Gamma, x:A \wp B \xrightarrow{x(x')} \Gamma, x':A, x:B \\
\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}} \text{PAR}_1 \quad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'} \text{PAR}_2 \quad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{ll'} \mathcal{G}' \mid \mathcal{H}'} \text{SYN} \\
\frac{\mathcal{G} \mid \Gamma, x:A^\perp \mid \Delta, y:A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x:A^\perp \mid \Delta', y:A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'} \text{RES} \quad \frac{\mathcal{G} \mid x:1 \mid \Gamma, y:\perp \xrightarrow{x[]|y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma} 1\perp \\
\frac{\mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \wp B^\perp \xrightarrow{x[x']|y(y')} \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi} \otimes \wp
\end{array}$$

Fig. 6. π MLL, transition rules for typing environments.

The requirement that a process can only perform actions permitted by its typing environment is known as **Session Fidelity**. Just as the *proc* function established operational correspondence for processes, the *env* function establishes a correspondence for environments.

While the rules in Figure 6 define how environments can transition in isolation, the execution of a well-typed program requires the process and its environment to step in unison. This co-dependent relationship has processes only perform actions permitted by its typing environment, while the environment dynamically mirrors the physical control flow of the process, and is formally captured by the property of **Session Fidelity**. Just as the *proc* function established operational correspondence for the untyped calculus, the *env* function establishes this exact behavioral correspondence for the typing contexts.

THEOREM 2.11 (SESSION FIDELITY). *Let $\mathcal{D}$ be a valid typing derivation such that $\text{env}(\mathcal{D}) = \mathcal{G}$ and $\text{proc}(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition*

$$\mathcal{G} \xrightarrow{l} \mathcal{G}',$$

and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 2.12 (Session Fidelity in Execution). Recall the process derivation from Example 2.5, typed by the environment $\mathcal{G} = v:1, w:\perp \mid y:1$. The process begins by performing an observable input action on w . By Session Fidelity, the environment must faithfully mimic this action, consuming the corresponding type:

$$v:1, w:\perp \mid y:1 \xrightarrow{w()} v:1 \mid y:1$$

The resulting derivative of the process is now typed by this new environment. At this intermediate stage, if we were to view the environment LTS in isolation, the pure semantics would theoretically allow an output on y to fire immediately, as $y:1$ is an available, unblocked resource.

However, Session Fidelity dictates a strict co-dependence. The environment merely mirrors the process, and the underlying process term strictly guards the $y[]$ action behind a prefix, requiring a prior τ -synchronisation. Therefore, the environment must wait. When the process finally resolves its internal communication, the environment mimics it via the τ -reflexivity axiom, remaining structurally unchanged:

$$v:1 \mid y:1 \xrightarrow{\tau} v:1 \mid y:1$$

Only after this synchronisation unblocks the continuation can the process perform the $y[]$ action, which the environment once again mirrors by consuming the final resource:

$$v : 1 \mid y : 1 \xrightarrow{y[]} v : 1$$

Remark 2.13 (The Mechanisation Gap). Session Fidelity is the ultimate touchstone for validating the safety of a session-typed language. However, verifying this property in a proof assistant exposes a stark gap between paper mathematics and mechanised logic. On paper, the invariant that an environment *dynamically mirrors* a process is highly intuitive, and transitions are easily massaged using structural equivalences and implicit variable conventions. In a machine, however, formally proving that types are consumed correctly requires a rigorous, algorithmic representation of bound variables and explicit freshness. The theoretical elegance of the environment LTS cannot be mechanised without first establishing a concrete architectural foundation for these variables, which dictates the design choices discussed in the following chapters.

2.8 Managing Bound Variables

Having established the operational semantics of π LL and the ultimate goal of verifying Session Fidelity, we must address the treatment of bound variables and freshness, a central technical challenge that underpins the entire formalisation.

As we have seen, many constructs in π LL introduce scoped identifiers. Restriction binds pairs of session endpoints, input constructs bind received channel names, polymorphic communication binds type variables, and duplication constructs introduce fresh channel endpoints. These bindings appear throughout both the process syntax and the labelled transition system.

The concrete choice of bound and introduced names is semantically irrelevant provided the binding structure is preserved. For example, renaming a restricted channel does not change the behaviour of a process. However, reasoning formally about substitutions, transitions, and typing derivations requires these names, scopes, and freshness conditions to be tracked explicitly.

This principle is formalised as α -equivalence, which identifies terms that differ only in the names of their bound variables. For instance, the following two processes are α -equivalent:

$$\nu ab(a[].0 \mid b().x[].0) =_{\alpha} \nu zw(z[].0 \mid w().x[].0)$$

The operational semantics of π LL relies on α -equivalence using $\text{rule } =_{\alpha}$ to rename bound variables on the fly, ensuring that restricted scopes and input prefixes do not accidentally capture free names as processes evolve.

Substitution raises this exact issue. Consider the substitution where the free name x is replaced by z in the right-hand process from the previous example:

$$\nu zw(z[].0 \mid w().x[].0)$$

Here z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z . Before the substitution can proceed, and because restriction binds session endpoints as pairs, both the bound z and w must be α -renamed to fresh names, such as to a and b . Similarly, any binder within the process that would capture the substituted name must also be renamed before substitution continues.

In paper-based presentations this is commonly handled by *Barendregt's variable convention*, which assumes bound variables are always chosen distinct from free variables and one another whenever convenient. This makes proofs readable but remains an informal meta-level assumption.

A proof assistant requires every freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named variables therefore introduces substantial proof overhead for capture avoidance and α -equivalence reasoning. To

avoid this, the formalisation adopts a *locally nameless* representation of binding, which eliminates α -equivalence as a proof obligation by construction. This is the subject of the following chapter.

3 Binding Architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 3.1), then introduce the locally nameless representation and its core operations (Sections 3.2 and 3.3), and conclude with co-finite quantification (Section 3.4), which governs how inductive definitions interact with binders.

3.1 Design Goals and the Approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

- (1) **Eliminate α -equivalence overhead.** α -equivalent terms should be represented identically, reducing α -equivalence reasoning to definitional equality within the proof assistant.
- (2) **Preserve human readability.** Terms should remain explicit and concrete, keeping substitution and freshness reasoning close to their paper-based presentation.
- (3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt’s convention or unverified freshness assumptions; capture avoidance and freshness conditions should instead be enforced by construction.

Although both channels and type variables involve binding, they exhibit fundamentally different operational behaviour. Session channels participate in dynamic communication and substitution across concurrent processes, while polymorphic type variables remain statically scoped within typing derivations. The formalisation therefore adopts different binding strategies for these two classes of identifiers while maintaining a common underlying binding architecture.

To achieve these goals, we adopt a locally nameless binding architecture for the calculus [Charguéraud 2012], separating free identifiers from bound occurrences. Bound variables are represented using indices, causing α -equivalent terms to coincide structurally, while free variables remain explicit and readable.

This separation simplifies substitution by restricting it to free variables while leaving bound indices untouched, thereby avoiding accidental capture by construction. As a result, many proofs that traditionally require substantial renaming infrastructure reduce instead to structural recursion and index manipulations.

The remainder of this chapter develops the theoretical machinery underlying this representation.

3.2 The Locally Nameless Representation

The locally nameless representation, as presented by Charguéraud [2012], separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

- (1) **Bound variables** are represented using *De Bruijn indices* [de Bruijn 1972], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α -equivalent terms become structurally identical.
- (2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of π LL.

LN Syntax for Types. In the original presentation of πLL , polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as follows:

$$\begin{aligned} T &::= \text{free}(x) \mid \text{bound}(k) && (\text{where } x, k \in \mathbb{N}) \\ A, B &::= \cdots \mid \exists A \mid \forall A \mid X \mid X^\perp && (\text{where } X \in T) \end{aligned}$$

Here, a free type variable carries a unique identifier x , while a bound type variable carries a De Bruijn index k . For example, the polymorphic type $\forall X. \exists Y. (X \wp Y)$ is structurally represented without names using the new syntax as $\forall. \exists. (\text{bound}(1) \wp \text{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN Syntax for Processes. Similarly, the standard process syntax displayed in [Section 2.2](#) is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[\cdot].P$ and $x(\cdot).P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$\begin{aligned} \text{Channel} &::= \text{free}(x) \mid \text{bound}(k) && (\text{where } x, k \in \mathbb{N}) \\ P, Q &::= \cdots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet, \bullet).P \mid x(\diamond).P && (\text{where } x \in \text{Channel}) \end{aligned}$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\nu(\bullet, \bullet).P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P , namely 0 and 1. For communication prefixes like $x(\bullet).P$, the payload is implicitly bound as index 0 within P , while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture elegantly resolves α -equivalence, moving between abstracted and instantiated forms requires specialised operations. To execute a substitution or evaluate a process across a binder, we must formally define how to open and close these terms.

3.3 Opening and Closing Binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [[Charguéraud 2012](#)].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k , then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k+1$, while traversing a double-name binder like $\nu(\bullet, \bullet).P$ would increment it to $k+2$.

When the recursion reaches a bound variable $\text{bound}(n)$, the index n is compared against the current depth k . If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\text{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

Crucially, when the opening operation encounters an existing free variable $\text{free}(y)$, it simply ignores it. This guarantees that opening is entirely capture-avoiding by construction. In the operational semantics, when a single binder like an input prefix $z(\bullet).P$ receives a channel x , the continuation P is opened as $\{0 \mapsto x\}P$. For a restriction $\nu(\bullet, \bullet)P$, the dual endpoints x and y are opened sequentially as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (Abstraction). Closing is the exact inverse of opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k , denoted $\{k \leftarrow x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\text{free}(y)$, it checked if $y = x$. If they match, the free name is replaced by $\text{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k .

Locally Closed Terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\text{bound}(0)[].\mathbf{0}$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To filter out these meaningless terms, the locally nameless architecture introduces the predicate of being *locally closed*, denoted $lc(P)$. A term is locally closed if and only if all of its De Bruijn indices are properly bound by an enclosing constructor.

The locally closed predicate excludes terms containing dangling indices and therefore characterises the well-formed syntax of the calculus. Establishing this predicate is not only a static validation step, but rather it forms a key invariant throughout the mechanisation, which guarantees the soundness of the calculus. Generally, any subsequent definition, such as variable substitution, structural congruence, or operational semantics, must be mathematically proven to *preserve* the locally closed property. In practice, this is achieved through the *regularity of typing*, which requires proving that all valid typing derivations inherently produce locally closed terms. Consequently, operations proven to preserve typing will naturally carry this invariant as a structural guarantee.

3.4 Co-finite Quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the final theoretical hurdle is defining how inductive relations, such as typing judgements and labelled transitions, operate across binders.

In paper-based proofs, when a rule requires opening a binder, Barendregt's variable convention allows the author to simply state: *pick a fresh name* x . In a proof assistant, this informal generation of a fresh name must be encoded into the constructors of the inductive relations. Two naive approaches to bind this fresh name, both of which introduce severe mechanical flaws [Aydemir et al. 2008; Charguéraud 2012]:

- (1) **Existential Quantification** ($\exists x \notin f(P)$): The rule requires the property to hold for at least one fresh name. While this makes proof construction straightforward, it yields weak induction hypotheses, since induction only provides the property for one particular fresh name chosen during the original derivation.

- (2) **Universal Quantification** ($\forall x$): The rule requires the property to hold for every name. This provides a strong induction hypothesis, but is too restrictive for proof construction, since the property cannot generally be shown for names that already occur free in the surrounding context.

To solve this dilemma, the locally nameless architecture employs **co-finite quantification** [Aydemir et al. 2008; Charguéraud 2012]. Rather than quantifying over one name or all names, the inductive rules quantifies universally over all names *except* those not in an arbitrary, finite set L . Formally, a rule requiring a fresh name uses the premise: $\forall x \notin L$.

By deferring the exact choice of the forbidden set L to the user at the time of proof, co-finite quantification achieves the best of both worlds. When constructing a proof tree, the user can instantiate L as the set of all free variables currently in the environment, process, and transition labels ($f(P) \cup f(\Gamma) \cup \dots$). This ensures the premise is trivially satisfiable. Conversely, when performing structural induction, the co-finite universal quantifier guarantees that the induction hypothesis applies to *any* fresh name the user could possibly need, provided they pick one outside the finite set L .

Application to π LL Types. Although type variables are represented using the locally nameless separation of free and bound constructors, their metatheory is handled using depth-indexed reasoning rather than cofinite quantification. Because type variables are static and do not dynamically communicate across parallel scopes like channels do, managing freshness sets L for them introduces unnecessary proof overhead.

Instead, the system evaluates polymorphic typing rules using a purely depth-indexed approach, using a De Bruijn shifting operator. We define the shifting of a type A by a distance d at a cutoff depth c , denoted mathematically as $\uparrow_d^c A$. Using this infrastructure, consider the typing rule for the universal type input ($\forall.B$):

$$\frac{n+1 \vdash P :: \uparrow_0^1 \Gamma, x:B}{n \vdash x(\diamond).P :: \Gamma, x:\forall.B} \forall_{\text{DB}}$$

Rather than opening the binder with a fresh explicit name, the typing judgment simply increments a depth counter $n+1$, legally bringing index 0 into scope. To maintain correct index distances, the typing environment Γ is explicitly shifted by a distance of 1 at cutoff 0 ($\uparrow_0^1 \Gamma$), incrementing all free type indices within the context to prevent accidental capture.

Application to π LL Channels. This co-finite quantification strategy fundamentally reshapes the inductive definitions for process channels. For example, consider the typing rule that introduces the prefix $x(\bullet).P$. Under a paper-based presentation, the premise implicitly requires a fresh session name y . Under our mechanised architecture, the rule uses co-finite quantification and the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x:B, y:A}{n \vdash x(\bullet).P :: \Gamma, x:A \wp B} \wp_{\text{LN}}$$

Here, the rule states that the process is well-typed if, for any fresh channel variable y outside some finite set of forbidden variables L , the opened process P is well-typed under the environment extended with the opened payload. By utilizing this mixed evaluation strategy, co-finite quantification for dynamic channels, and depth-indexed evaluation for static types, the architecture minimizes bureaucratic renaming overhead while maintaining a unified underlying syntax.

With this mathematical foundation established, the conceptual theory of π LL is fully prepared for implementation. The following chapter details how this locally nameless architecture is concretely mechanised in the Lean 4 proof assistant.

The corresponding Lean definitions for channels, opening and closing, and selected local-closure predicates are included in ??; the full development is available in the accompanying codebase.

4 Formalisation

This section describes the mechanisation of π LL in Lean 4, bridging the theoretical foundations developed in Sections 2 and 3 and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the metatheoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from Montesi and Peressotti [2021].

4.1 Model

The `Model` namespace formalizes the static structure of the full π LL calculus and is organized around the layered architecture introduced in Section 2. The mechanisation proceeds bottom-up: session types, defined in `STypes/`, provide the binding structure used by processes in `Processes/`, processes are then typed relative to environments and hyperenvironments from `Environments/` and `HyperEnvironment/`, finally, typing derivations and their metatheoretic infrastructure are collected in `Judgement/`. This includes inversion lemmas, linearity properties, substitution principles, and the co-finite freshness machinery described in Section 3.4. Shared syntactic infrastructure used across these layers is provided by `Base.lean`.

4.1.1 Session Types. Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`. Following the locally nameless syntax established in Section 3.2, type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
| free (x : FTVar)
| bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a De Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces uninterpreted atomic types (`atom` and `atomDual`), representing protocol leaves not further decomposed by the session type grammar and providing the base cases for the inductive structure of the syntax. Table 3 illustrates a representative correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in Section 3.4. The constructors keep the bound De Bruijn index implicit, taking only the continuation body `A` as an argument (an underscore “`_`” has been appended to the constructor names to prevent namespace collisions with Lean’s built-in `forall` and `exists` keywords).

Duality. `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types -> Types
| .tensor A B => .parr (dual A) (dual B)
| .forall_ A => .exists_ (dual A)
| .one => .bot
| ...
```

Table 3. Comparison of standard, locally nameless, and Lean representations for π LL types.

Standard	Locally Nameless	Lean Constructor	Binds
$A \otimes B$	$A \otimes B$	tensor (A B : Types)	None
1	1	one	None
$\forall X.A$	$\forall.A$	forall_ (A : Types)	1 type
$\exists X.A$	$\exists.A$	exists_ (A : Types)	1 type
X	X	var (v : TVar)	None

The fundamental property that duality is an involution, $(A^\perp)^\perp = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\text{lc} \iff A^\perp.\text{lc} \quad \uparrow_d^c (A^\perp) = (\uparrow_d^c A)^\perp \quad B^\perp\{A/k\} = (B\{A/k\})^\perp$$

Local closure. `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types → Prop`, where the depth parameter k tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. By contrast, a bound variable is only locally closed if its De Bruijn index i is strictly less than the current depth k , ensuring it points to an enclosing scope:

```
def TVar.lc : Nat -> TVar -> Prop
| _, .free _   => True
| k, .bound i  => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat -> Types -> Prop
| _, .one       => True
| k, .var v     => TVar.lc k v
| k, .tensor A B => A.lc k ∧ B.lc k
| k, .forall_ A  => A.lc (k + 1)
| k, .exists_ A  => A.lc (k + 1)
| ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0. This is registered as the abbreviation `Types.lc_0`.

Shifting. `STypes/Shift.lean` defines shifting following the standard De Bruijn shifting operation introduced in [Section 3.4](#). The operation traverses the syntax tree structurally using a cutoff depth d and a shift amount c .

For standard connectives, shifting distributes recursively across subterms, while base constructors such as one and bot remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types -> Types
| .var v      => .var (v.shift d c)
| .tensor A B => .tensor (A.shift d c) (B.shift d c)
| .forall_ A  => .forall_ (A.shift (d + 1) c)
| t          => t
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the De Bruijn index i against the cutoff depth d . Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by c to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar -> TVar
| .bound i => if i < d then .bound i else .bound (i + c)
| .free n  => .free n
```

Substitution. `STypes/Subst.lean` defines substitution as the replacement of a target De Bruijn index k in a host type B by a payload type A , denoted $B\{A//k\}$. At the variable level, the function compares a bound De Bruijn index i with the target depth k . If $i = k$, the target index has been reached and the variable is replaced by A . If $i < k$, the variable is bound by an inner scope and is left unchanged. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to k , such indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.bound i) =>
    if i == k then A
    else if i > k then .var (.bound (i - 1))
    else .var (.bound i)
```

The operation is lifted to the full `Types` syntax by structural recursion. For ordinary connectives, substitution distributes across subterms. When descending into a polymorphic binder such as `forall_`, the target depth is incremented to $k + 1$, and the payload type A is shifted by one. This prevents the free indices in A from being captured by the newly entered binder:

```
| .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))
```

Notation. `STypes/Notation.lean` introduces notation mirroring the mathematical presentation from [Section 2.3](#). The only divergence is that exponentials are written `!!A` and `??A` to avoid clashes with Lean syntax.

The full inductive definition of `Types` and all lemmas from `STypes/Lemmas.lean` are listed in [??](#).

4.1.2 Processes. Processes are defined in `Processes/Basic.lean` as the inductive type `Proc`. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the

supremum of any finite set of existing names (mechanised in `Processes/Fresh.lean`). Conversely, bound channels are represented using De Bruijn indices, encoding their relative distance to the corresponding binder.

```
inductive Channel : Type where
| free (x : FPNName)
| bound (x : BPNName)
```

As discussed in [Section 3.2](#), binding constructs in the process syntax no longer carry explicit payload variables. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the inductive `Proc` type illustrates this abstraction:

Standard	Locally Nameless	Lean Constructor	Binds
$x[y].P$	$x[\bullet].P$	<code>tensor</code> (x : Channel) (P : Proc)	1 channel
$x(y).P$	$x(\bullet).P$	<code>parr</code> (x : Channel) (P : Proc)	1 channel
$\nu xy.P$	$\nu(\bullet, \bullet).P$	<code>cut</code> (P : Proc)	2 channels
$x(X).P$	$x(\diamond).P$	<code>input</code> (x : Channel) (P : Proc)	1 type
$x[\text{DUP}](x').P$	$x[\text{DUP}](\bullet).P$	<code>duplicate</code> (x : Channel) (P : Proc)	1 channel

Notice that `cut` takes only a continuation P with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of P . Similarly, for constructs like `tensor`, `parr`, and `input`, the subject channel x over which the interaction occurs is kept explicit, while the transmitted payload is anonymized into the continuation.

Opening and closing. `Processes/OpenClose.lean` defines opening and closing operations following the locally nameless principles established in [Section 3.3](#). In the theoretical presentation, opening a process at depth k with x is denoted as $\{k \mapsto x\}P$, and conversely, abstracting a specific name back into a bound index is denoted as $\{k \leftarrow x\}P$.

To mechanise this in Lean, we denote explicit free channels as $\#x$ (mapping to `Channel.free x`) and introduce custom notation for these substitution operations. Expressions such as $P((k|\#x))$ and $P((k|\#x, \#y))$ denote the opening of a process by instantiating the k -th (and $k + 1$ -th) bound indices with free channels. Conversely, $P\langle\langle k|\#x \rangle\rangle$ and $P\langle\langle k|\#x, \#y \rangle\rangle$ denote the closing of a process by abstracting x and y back into De Bruijn indices. For convenience, the depth parameter can be omitted when operating at the innermost binder ($k = 0$), simplifying the notation to $P((\#x, \#y))$ and $P\langle\langle \#x, \#y \rangle\rangle$.

Because binders are represented positionally, the depth parameter k increments according to the number of channels bound by each constructor. For standard communication prefixes like `tensor`, `parr`, and the server `duplicate` operation, exactly one channel is bound, incrementing k by one. The restriction constructor (`cut`) simultaneously binds two dual endpoints, requiring k to increment by two. Notably, the `input` constructor binds a type variable rather than a channel. Consequently, it does not affect the channel depth parameter k .

```
def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P => .tensor (x.open u k) (P.open u (k + 1))
  | .parr x P   => .parr (x.open u k) (P.open u (k + 1))
  | .cut P      => .cut (P.open u (k + 2))
  | .input x P  => .input (x.open u k) (P.open u k)
```



```
| ...
```

At the leaves of the syntax tree, this recursive traversal delegates to the base case defined by `Channel.open`. For bound channels, the function evaluates the De Bruijn index i against the target depth k , replacing it with the instantiated channel v upon an exact match. Explicit free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i
  | c => c
```

For restriction the `HasOpenTwo` typeclass from `Base.lean` is instantiated by opening sequentially: first replacing index $k + 1$ with the second endpoint v , then replacing index k with the first endpoint u :

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close` and `Channel.close` define the corresponding closing operation. Used primarily when abstracting explicit free names into new scopes, `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the exact same depth-incrementing rules for single and double binders. When the recursion reaches a leaf node, it delegates to `Channel.close`. If the leaf is a free channel (`FPName`), the function compares it against the target name x , upon a match, the explicit name is *abstracted* into a bound De Bruijn index at the accumulated depth k :

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i
```

Local closure. `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: k tracks the depth of channel binders, while n tracks the depth of type binders. The type depth is needed when a process embeds a type, for instance as the payload of an output prefix, since the embedded type must be checked relative to the current type-binder depth.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth k , while type binders increase the type depth n . For example, tensor binds a single continuation channel, incrementing the depth to $k + 1$, whereas cut binds two dual endpoints simultaneously, incrementing the depth to $k + 2$. Conversely, input binds a type variable, leaving k unchanged while incrementing n to $n + 1$. Constructors introducing no binders, leave both counters unchanged:

```
def Proc.lc : Nat -> Nat -> Proc -> Prop
| _, _, .nil      => True
| k, n, .one x P   => x.lc k ∧ P.lc k n
| k, n, .tensor x P => x.lc k ∧ P.lc (k + 1) n
| k, n, .cut P     => P.lc (k + 2) n
| k, n, .input x P => x.lc k ∧ P.lc k (n + 1)
```


| ...

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning $\text{Proc.lc } 0 \ 0$ holds. This is registered as the abbreviation Proc.lc_0 .

Free names and freshness. Processes/Names.lean defines $\text{Proc.f} : \text{Proc} \rightarrow \text{Finset FPNName}$ to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Since locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations. Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations:

```
def Proc.f : Proc -> Finset FPNName
| .cut P      => P.f
| .par P Q    => P.f ∪ Q.f
| .link x y   => x.f ∪ y.f
| .tensor x P => x.f ∪ P.f
| .one x P    => x.f ∪ P.f
| .nil       => {}
| ...
```

The resulting finite set is used by Processes/Fresh.lean to generate names outside a forbidden context. Given a finite set L of forbidden names, a fresh name is computed as $\max(L) + 1$. The lemmas `exists_one_fresh` and `exists_two_fresh` guarantee one or two distinct names outside L , respectively. These lemmas provide witnesses for the co-finite quantification premises used throughout the typing and transition rules, in particular for restriction, where two fresh and distinct endpoints must be introduced simultaneously.

Substitution. The process syntax supports two substitution operations, reflecting its two-sorted binding structure. Name substitution is defined in Processes/SubstNames.lean as the function $\text{Proc.substNames} (R \ T : \text{FPNName}) : \text{Proc} \rightarrow \text{Proc}$, replacing all explicit free occurrences of the target name T with the replacement name R . Since bound channels are represented by De Bruijn indices, name substitution only acts on explicit free channel leaves and leaves binders untouched. Consequently, the operation is capture-avoiding by construction and does not require α -renaming or additional freshness side conditions.

Type substitution is defined in Processes/SubstTypes.lean as $\text{Proc.substTypes} (A : \text{Types}) (k : \text{Nat}) : \text{Proc} \rightarrow \text{Proc}$, substituting A for the De Bruijn type index k throughout a process. Whenever the traversal encounters an embedded type, such as the payload of an output prefix, it delegates to the type-level substitution operation `Types.subst`. When crossing an input prefix, which binds a type variable, the target depth is incremented and the substituted type is shifted, mirroring the binder case of `Types.subst` from [Section 4.1.1](#):

```
def Proc.substTypes (A : Types) (k : Nat) : Proc -> Proc
| .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of this representation is that no analogous channel shifting operation, such as `shiftNames`, is required. In a pure De Bruijn encoding, variables are represented as relative indices and must be shifted when crossing binders. Free channels, however, are represented as global identifiers rather than relative positions, so their representation is unaffected by binder depth. This

avoids structural adjustment of channel names during substitution and simplifies the process-level infrastructure.

Structural Properties. `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. This includes commutation lemmas for interacting operations (e.g., `open_substNames_comm`, showing that opening commutes with name substitution), preservation properties for local closure (e.g., `lc_of_open`), and lemmas relating binders to free-name analysis (e.g., `f_close`, proving that closing removes a free name from the resulting term).

Of particular importance are the commutation lemmas between opening and substitution: `open_substNames_comm` establishes that opening and name substitution commute, and the corresponding lemma for type substitution ensures that substituting under a binder produces the same result regardless of order. These properties are essential in the typability proof, where the co-finite witnesses produced by the typing rules must be reconciled with the concrete names introduced by the process transitions.

These results provide the core infrastructure required for the transition semantics developed in subsequent chapters.

For completeness, the full inductive definition of `Proc` and all lemmas from `Processes/Lemmas.lean` are listed in ??.

4.1.3 Environments. Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation. Theoretical environments are unordered and duplicate-free, suggesting `Finset` as a natural implementation. However, since `Finset` is implemented as a quotient over lists, it complicates structural induction, pattern matching, and inversion over environments. This proved inconvenient in a development where typing derivations are frequently inverted and typing environments must be rearranged explicitly. A second attempt using association lists (`AList`) encountered similar proof-engineering difficulties. The final design therefore uses plain lists and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction and simpler interaction with Lean’s tactic machinery.

Recovering the PCM structure. Merging two environments is implemented as list concatenation, making it a simple structural, but total, function:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 ", " => Env.merge -- uses \glq to avoid clashing with the actual comma
```

Because lists do not naturally form a Partial Commutative Monoid (PCM) but are associative by definition, we only need to formally recover commutativity.

Since list concatenation is associative definitionally but not commutative, the commutative structure of environments is recovered by reasoning up to permutation. This is achieved using Lean’s built-in `List.Perm` relation, instantiated through a custom `HasPerm` typeclass and denoted by the infix relation $\Gamma \sim \Delta$.

The essential monoidal laws, including commutativity, associativity, and left and right identity for $\emptyset$,¹ are established as:

```
lemma Env.merge_comm (Γ Δ : Env) : Γ, Δ ~ Δ, Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ, Δ, Ξ = Γ, (Δ, Ξ)
```

The partiality of environment merging is enforced by explicit predicates rather than by making `Env.merge` a partial function. `Env.names` extracts the domain as a `Finset`, enabling Lean’s built-in `Disjoint` typeclass. Internal linearity (no channel typed twice) is enforced by `Env.Nodup`:

```
def Env.names (Γ : Env) : Finset FPNName := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The relationship between linearity, disjointness, and merging of typing environments is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear if and only if both component environments are individually linear and mutually disjoint.

Server usability. The typing rule for server replication requires all dependency channels to carry exponential request types. This side condition is captured by `Env.serverUsable`, which asserts that every type in an environment satisfies `Types.isServerUsable`, i.e. has the form $?B$.

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀p, p ∈ Γ -> p.snd.isServerUsable
prefix:max "?e" => Env.serverUsable
```

Lifted operations. Since environments contain types, the locally nameless operations from [Section 4.1.1](#) are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list. The file `Environment/Lemmas.lean` proves that these operations preserve the names, disjointness, nodup, and permutation properties of environments.

The full definitions and lemmas are listed in ??.

4.1.4 Hyperenvironments. Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
```

As for environments, merging is implemented as list concatenation, using $\emptyset$ as the unit and $\mathcal{G}|_h\mathcal{H}$ as notation for composition. Associativity and unit laws therefore follow from the underlying list structure. However, recovering the commutative structure requires more than the ordinary list permutation relation, since hyperenvironments are lists whose elements are themselves environments considered modulo permutation.

Deep permutation. For environments, Lean’s `List.Perm` relation is sufficient, since it allows arbitrary reordering of channel-type pairs. For hyperenvironments, however, permutation must operate at two levels: it must allow reordering of component environments, and it must also respect permutation equivalence inside each component environment.

For example, consider the hyperenvironment containing a single component environment $[x:A, y:B]$. Ordinary `List.Perm` treats this component environment as an opaque element, so it

¹Using $\emptyset$ rather than the literal `[]` requires explicit rewrite lemmas for terms such as $\emptyset, \Gamma$. If `[]` were used directly, `[]`, Γ would reduce by definitional equality, since `Env.merge` abbreviates `List.append`.

does not identify it with $[y:B, x:A]$. The custom relation `HyperEnv.Perm` instead permits component environments to be replaced by permutation-equivalent environments.

```

inductive HyperEnv.Perm : HyperEnv -> HyperEnv -> Prop where
  | nil : Perm [] []
  | cons : {Γ Δ : Env} {G H : HyperEnv} : Γ ~ Δ -> Perm G H -> Perm (Γ :: G) (Δ :: H)
  | swap : {Γ Δ : Env} {G : HyperEnv} : Perm (Γ :: Δ :: G) (Δ :: Γ :: G)
  | trans {G H J : HyperEnv} : Perm G H -> Perm H J -> Perm G J

```

The key constructor is `cons`. Unlike ordinary list permutation, it does not require the head environments to be syntactically identical, it only requires them to be permutation-equivalent. Thus `HyperEnv.Perm` is a deep permutation relation, propagating equivalence both across the ordering of component environments and within the environments themselves.

Well-formedness. Hyperenvironment well-formedness is decomposed into two predicates. The predicate `HyperEnv.Nodup` requires each component environment to be internally linear, while `HyperEnv.PairwiseDisjoint` requires distinct component environments to have disjoint domains:

```

def HyperEnv.Nodup (G : HyperEnv) : Prop :=
  ∀ Γ, Γ ∈ G -> Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (G : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint G

```

The distinction is important because the two predicates rule out different violations of linearity. For example, assuming x, y , and z are distinct names:

- $[x:A] \mid_h [y:B, z:C]$ satisfies both predicates.
- $[x:A, x:B] \mid_h [y:C]$ violates `Nodup`, since x appears twice within one component environment.
- $[x:A] \mid_h [x:B, y:C]$ violates `PairwiseDisjoint`, since x appears in two distinct components.

Together, these predicates enforce global linearity: every channel appears at most once across the whole hyperenvironment.

Lifted operations. As with environments, type shifting, name substitution, and type substitution are lifted pointwise over the list of component environments. The file `HyperEnvironment/Lemmas/Basic.lean` proves that these operations preserve names, disjointness, pairwise disjointness, and the deep permutation relation. For example, preservation of permutation under shifting is proved by induction on `HyperEnv.Perm`, using the corresponding environment-level permutation lemma in the `cons` case.

The full definitions and lemmas are listed in ??.

4.1.5 Typing Judgements. The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```

inductive Typing : Nat -> Proc -> HyperEnv -> Prop

```

We write $n \vdash P :: G$ for a derivation of this predicate. The index n tracks the current type-binder depth, while the remaining indices record the process being typed and its associated typing environment. This intrinsic indexing is later used by the erasure maps in the semantics and metatheory.

Type-depth indexing. The type-depth index is relevant whenever the derivation crosses a polymorphic binder. For example, the `forall_` constructor increments the depth from n to $n + 1$ and shifts the surrounding environment to keep embedded type indices in scope:

```
| forall_ :
  Typing (n + 1) P [x:B :: Γ+] ->
  Typing n (λx((◊)).P) [x:∀.B :: Γ]
```

The shift Γ^+ abbreviates shifting the type indices in Γ by one, mirroring the binder-sensitive shifting used by type substitution in [Section 4.1.1](#).

Exchange and explicit side conditions. Because environments and hyperenvironments are represented as lists, the unordered and partial structure of the theoretical hyperenvironment is recovered explicitly. The typing relation includes exchange rules for reasoning up to permutation equivalence:

```
| exchange_env   : Typing n P (Γ :: G) -> Γ ~ Δ -> Typing n P (Δ :: G)
| exchange_hyper : Typing n P G -> G ~ H -> Typing n P H
```

The exchange rules recover the unordered character of the paper presentation without quotienting environments or hyperenvironments. The complementary issue is partiality: although merging is implemented as total list concatenation, typing rules carry explicit disjointness and freshness premises to ensure that only valid linear compositions are formed. For example, `mix` requires the two hyperenvironments to be disjoint, while prefix rules such as `tensor` and `parr` require freshness premises ensuring that subject channels do not already appear in the continuation environment.

The same design principle reappears later in the operational semantics, where transitions must also respect permutation equivalence of environments and hyperenvironments.

Co-finite quantification. Binding rules use co-finite quantification as described in [Section 3.4](#). For example, the `parr` constructor types an input prefix by requiring the opened continuation to be well typed for every fresh payload name outside a finite set L :

```
| parr (hF : x ∉ Γ.names) (L : Finset FPNName) :
  (∀y ∉ L, Typing n (P((#y))) [y:A :: x:B :: Γ]) ->
  Typing n (x((●)).P) [x:A ⋈ B :: Γ]
```

Here, the process carries no explicit payload name; the bound occurrence in the continuation is instantiated by opening P with the fresh name y . The premise `hF` enforces that the subject channel x is not already present in the continuation environment. Rules binding multiple names, such as `cut`, follow the same pattern but quantify over two distinct fresh names.

Structural invariants. `Judgement/Names.lean` establishes the central name invariant for the typing relation, named `Typing_f_eq_names`. It states that, for any well-typed process, the free names of the process coincide exactly with the domain of its hyperenvironment.

$$n \vdash P :: G \implies P.f = G.names$$

This ensures exact correspondence between process free names and typed environment entries: no free process channel is left untyped, and no environment entry is unused.

Building on this, `Judgement/Properties/Linearity.lean` proves `Typing_preserves_linearity`: any well-typed hyperenvironment is pairwise disjoint and all component environments have no duplicates. Together these results guarantee that typing derivations can only be constructed for well-formed, linear contexts.

The full inductive definition of `Typing` is listed in ??.

4.2 Semantics

The dynamic behaviour of the π MLL fragment is formalised in the Semantics/ namespace. Following the erasure structure described in Section 2.5, the development contains three related transition systems: a typed transition system over typing derivations, and two projected systems describing the induced process and environment behaviour.

- **TypingStep_m**: the primary, resource-aware LTS over typing derivations (JudgementStep/Basic.lean);
- **ProcStep_m**: the process-level LTS obtained by forgetting typing resources (ProcStep/Basic.lean);
- **EnvStep_m**: the resource-level LTS describing the corresponding evolution of hyperenvironments (EnvStep/Basic.lean).

Labels. Transition labels are formalised in Labels.lean. Although the operational semantics mechanised here target π MLL, the underlying action vocabulary is defined for the full π LL calculus to support future extensions.

The core action type, Act, captures observable communication events, including empty communication, channel transmission, type instantiation, and server interaction. Selection and server actions are parameterised by the auxiliary datatype Mu. The Lbl inductive then extends observable actions with the silent transition τ , explicit session-link labels, and a parallel constructor combining two observable actions directly, reflecting the synchronized action labels used by the parallel rules.

```

inductive Act : Type
| one      (x : Fpname)
| tensor   (x y : Fpname)
| output   (x : Fpname) (A : Types)
| muBrack  (x : Fpname) ( $\mu$  : Mu)
| ...

inductive Lbl : Type
| tau
| link (x y : Fpname)
| act  (p : Act)
| par  (l l' : Act)

```

Each label defines computable sets of free names (Lbl.f) and introduced names (Lbl.i), which are used in the side conditions for the rules **PAR₁**, **SYN** and **PAR₂**. To preserve alignment with the theoretical presentation, we use the HasBracket and HasParen typeclasses for Act and Lbl to allow the notation for process prefixes to be reused uniformly for labels as well. For instance, $x[[y]]$ denotes both the output process prefix and the corresponding output label.

4.2.1 The Typed Transition System. The primary LTS, TypingStep_m, is a relation between typing derivations. Its constructors mirror the operational rules shown in Figure 2, but make the type-theoretic content of each step explicit: every transition carries a typing derivation as its source and produces a typing derivation as its target. The system is therefore intrinsically typed.

```

inductive TypingStepm :
{n : Nat} → {G : HyperEnv} → {P : Proc} → Typing n P G →
Lb1 → {n' : Nat} → {G' : HyperEnv} → {P' : Proc} → Typing n' P' G' → Prop

```

As established previously, both environment and hyperenvironment composition are formalised using total list appends. Consequently, the explicit disjointness proofs required by the static Typing

relation to safely mix environments must propagate directly into the dynamic TypingStep_m relation. For instance, the par_1 constructor requires explicit hypotheses (hD1 and hD2) to guarantee that the hyperenvironments remain strictly disjoint both before and after the transition:

```

| par1
  (hD1 :  $\mathcal{G}.\text{disjoint } \mathcal{H}$ ) (hD2 :  $\mathcal{G}'.\text{disjoint } \mathcal{H}$ )
  (h :  $\text{TypingStep}_m \mathcal{D} \ 1 \ \mathcal{D}'$ )
  (disj :  $1.i \cap Q.f = \emptyset$ ) :
  TypingStepm
    (Typing.mix hD1  $\mathcal{D} \ \mathcal{E}$ )
    1
    (Typing.mix hD2  $\mathcal{D}' \ \mathcal{E}$ )

```

The post-state disjointness proof hD2 is included explicitly as a convenience: while it is derivable from hD1, the freshness condition $i(I) \cap Q.f = \emptyset$, and the structural property that transitions only introduce names recorded by their label, keeping it as a premise avoids reconstructing this fact repeatedly in later proofs.

Remark 4.1. Strictly stepping typing derivations can expose structural artifacts that are usually hidden in the paper presentation. For example, in Figure 3, the intermediate derivation is typed by $\emptyset \mid y : 1$ rather than simply $y : 1$, because the MIX rule preserves the empty component contributed by the terminated left process. In Lean, this corresponds to a Typing.mix constructor containing a $\text{Typing.mix}_\emptyset$ sub-derivation. The result is valid, but reducing it to the canonical form requires applying the monoidal identity laws of the hyperenvironment explicitly, rather than by definitional equality. The final $\equiv$ step in the figure, which additionally uses process congruence to identify $\mathbf{0} \mid \mathbf{0}$ with $\mathbf{0}$, is not currently automated because structural congruence has not been mechanised. This limitation is discussed further in Section 6.

4.2.2 Projected Transition Systems. The process and environment transition systems are defined to capture the two erasure views of a typed transition. ProcStep_m records the process behaviour obtained by forgetting typing resources, while EnvStep_m records the corresponding resource evolution. These relations are later connected to TypingStep_m by the simulation theorems in Section 5.1.

Process steps. ProcStep_m models the syntactic evolution of processes independently of typing resources. Unlike the typing relation, it is not indexed by a typing derivation and therefore does not use co-finite premises. Binder-opening rules instead choose concrete fresh names directly and record the required freshness as explicit side conditions. For example, the tensor rule opens the continuation with a concrete name y , requiring y to be fresh for both the subject channel and the continuation:

```

| tensor {P : Proc} {x y : FPNName} (hF :  $y \notin \{x\} \cup P.f$ ) :
  ProcStepm ( $\#x[[\bullet]].P$ ) ( $x[[y]]$ ) P( $\#y$ )

```

This creates a mismatch with the co-finite formulation of the typing rules. Thus ProcStep_m remains close to a standard concrete transition relation, while the additional freshness reconciliation is handled in the typability proof. A co-finite formulation of ProcStep_m would align more directly with the typing rules, but the present formulation keeps the erased LTS closer to a standard concrete transition relation.

Environment steps. The structural evolution of hyperenvironments, written $\mathcal{G} \xrightarrow{l} \mathcal{H}$, is formalised by EnvStep_m . Unlike the typed transition system, EnvStep_m deliberately contains only the resource

transformation described by the label. For example, an input action on an endpoint typed by $\perp$ consumes that resource: $\Gamma, x : \perp \xrightarrow{x()} \Gamma$. Similarly, an empty output on an endpoint typed by 1 evolves $x : 1$ to the empty environment. These rules describe how session types evolve as actions are performed, independently of any particular process term.

The relation does not independently enforce all disjointness and well-formedness conditions. Those properties are supplied by the typing derivation in `TypingStepm`. This separation keeps the projected resource semantics lightweight: the environment relation describes how resources change, while the typed semantics proves that these changes occur from well-formed linear contexts.

The following metatheory shows that these transition systems agree: typed transitions project to process and environment transitions, and process transitions from well-typed states can be lifted back to typed transitions.

5 Metatheory

This section describes the metatheoretic results connecting the typed, process, and environment transition systems defined in [Section 4.2](#). The development is contained in the `SessionFidelity/` namespace and focuses on session fidelity for the mechanised π MLL fragment.

5.1 Session Fidelity

Session fidelity states that well-typed processes evolve according to their session types, and that their associated linear resources evolve consistently with each process transition. The proof proceeds in two directions. First, erasure simulation shows that typed derivation steps project to valid process and environment transitions. Second, typability and subject reduction show that process transitions from well-typed processes can be lifted back to typed derivation steps and therefore preserve typing.

Erasure simulations. The paper establishes erasure as a strong bisimulation between lts_p and lts_d (Theorem 4.7 in [Montesi and Peressotti \[2021\]](#)), meaning the relation $\{(proc(\mathcal{D}), \mathcal{D}) \mid \mathcal{D} \in Der\}$ is closed in both directions. The mechanisation currently establishes the forward direction via `session_fidelity_procm`: every typed derivation step projects to a process step. The backward direction, which lifts process steps from well-typed processes to typed derivation steps, is captured by `typability_subject_reductionm`, but has not been packaged as an explicit bisimulation statement.

The files `ProcStep.lean` and `EnvStep.lean` establish the two projection results from typed derivation steps. Given a typed transition `TypingStepm  $\mathcal{D} \vdash \mathcal{D}'$` , `session_fidelity_procm` projects it to a process transition between $proc(\mathcal{D})$ and $proc(\mathcal{D}')$, while `session_fidelity_envm` projects it to an environment transition between $env(\mathcal{D})$ and $env(\mathcal{D}')$. The first result corresponds to the forward direction of the paper’s erasure theorem between lts_p and lts_d . The second corresponds to derivation-level session fidelity, Theorem 4.9.

```
theorem session_fidelity_procm
  { $\mathcal{D}$  : Typing n P  $\mathcal{G}$ } { $\mathcal{D}'$  : Typing n' P'  $\mathcal{G}'$ }
  (hStep : TypingStepm  $\mathcal{D} \vdash \mathcal{D}'$ ) :
  ProcStepm (proc  $\mathcal{D}$ )  $\vdash$  (proc  $\mathcal{D}'$ )
```

```
theorem session_fidelity_envm
  { $\mathcal{D}$  : Typing n P  $\mathcal{G}$ } { $\mathcal{D}'$  : Typing n' P'  $\mathcal{G}'$ }
  (hStep : TypingStepm  $\mathcal{D} \vdash \mathcal{D}'$ ) :
  EnvStepm (env  $\mathcal{D}$ )  $\vdash$  (env  $\mathcal{D}'$ )
```


These results establish that typed transitions project to the corresponding process and environment transitions. Both proofs proceed by induction on the typed transition derivation. Most cases are immediate constructor translations. The main exception is formed by the tensor and parr prefix cases, where the projected process and environment rules require explicit freshness and disjointness side conditions. These are recovered from the co-finite typing premises by instantiating the continuation with a sufficiently fresh auxiliary name and using the typing invariants `Typing_f_eq_names` and `Typing_preserves_linearity`. For the process projection, this also requires relating the free names of an opened process to those of the original continuation.

Typability. The main lifting theorem is stated in `Typability.lean`. It corresponds to the reverse direction of the erasure correspondence: given a process transition from a well-typed process, the transition can be lifted to the level of typing derivations.

This formulation differs from the paper presentation, which formulates erasure extensionally, using set-valued transition functions and equality of successor sets. In the Lean formalisation, transitions are represented as inductive relations, so the same correspondence is decomposed into relational projection and lifting directions. The theorem `session_fidelity_proc_m` gives the forward projection from typed derivation steps to process steps, while `typability_subject_reduction_m` gives the converse lifting direction for processes equipped with a typing derivation. This avoids defining successor sets explicitly in Lean and allows the proof to follow the induction principles generated by the transition relations.

```

theorem typability_subject_reduction_m
  (D : Typing n P G) (hPS : ProcStep_m P l P') :
  ∃ (G' : HyperEnv) (D' : Typing n P' G'),
    TypingStep_m D l D'

```

This theorem forms the technical core of the session fidelity argument. Its proof proceeds by induction on the process transition derivation. Each case inverts the source typing derivation to recover the typing rule compatible with the observed process shape, then constructs a target hyperenvironment and typing derivation for the resulting process.

The proof has two main technical obligations. First, it must reconcile the concrete freshness witnesses used by `ProcStep_m` with the co-finite premises of the typing rules. For example, a process transition fixes a concrete name introduced by a prefix, while the corresponding typing rule provides a derivation only for all names outside some finite set. Auxiliary *all fresh* lemmas bridge this gap by showing that the co-finite premise can be instantiated with the concrete name appearing in the process step. Second, in the communication and restriction cases, the proof must rearrange hyperenvironments to expose the components containing the communicating endpoints. In particular, synchronization under restriction requires opening the restricted process with fresh endpoints, applying the induction hypothesis to the opened body, and reconstructing the target hyperenvironment up to permutation.

The remaining incomplete case is `res`. In this case, the induction hypothesis yields a typed step for the opened body of the restriction. The missing step is to invert this typed transition to recover the post-state components corresponding to the restricted endpoints, then repackage them into the co-finite premise required by the outer cut typing rule. This hyperenvironment reconstruction remains the main outstanding proof obligation.

Subject reduction. Subject reduction is stated in `SubjectReduction.lean`. It says that if a well-typed process takes a process transition, then the target process is well typed under some evolved hyperenvironment, and that this hyperenvironment evolves according to the same label.

```

1569 theorem subject_reduction_m
1570   {n : Nat} {G : HyperEnv} {P P' : Proc} {l : Lbl}
1571   (D : Typing n P G) (hPS : ProcStep_m P l P') :
1572    $\exists G', \text{EnvStep}_m G l G' \wedge \text{Typing } n P' G'$ 

```

1573 The proof is a direct composition of the previous results: `typability_subject_reduction_m`
1574 lifts the process step to a typed derivation step, and `session_fidelity_env_m` projects that typed
1575 step to the corresponding environment transition. This realises the process-level session fidelity
1576 result corresponding to Corollary 4.10 of Montesi and Peressotti [2021].
1577

1578 *Implementation considerations.* In the mechanisation, several structural obligations that are
1579 implicit in the paper presentation become explicit. Since environment and hyperenvironment
1580 composition are implemented by total list concatenation, disjointness and freshness conditions
1581 appear as explicit premises in the typing and transition rules. Moreover, the process LTS uses
1582 concrete freshness witnesses, while the typing rules use co-finite quantification. This does not
1583 change the mathematical content of the semantics, but it introduces additional proof obligations in
1584 the typability theorem. Both issues are discussed further in Sections 6 and 7.

1585 *Interpretation.* Together, these results establish the structure of the session fidelity argument
1586 for the mechanised fragment, with one remaining gap. The erasure simulations are complete, and
1587 subject reduction follows once typability is fully established. The typability theorem is proved
1588 for all multiplicative cases except `res`. In this case, the proof must show that environments not
1589 involved in the transition are preserved, up to permutation, across restriction steps. The required
1590 infrastructure is largely in place, with the remaining work concentrated in the case analysis relating
1591 uninvolved parallel environments to the post-step hyperenvironment produced by the restricted
1592 transition. Once this case is closed, the full process-level session fidelity property follows for the
1593 mechanised π MLL fragment.

1594 The paper additionally establishes behavioural properties including bisimilarity, the diamond
1595 property, serialisation, non-interference, and readiness for π MLL. These results are not mechanised
1596 in the current formalisation; they are discussed in Section 7.
1597

1598 6 Discussion

1599 This section reflects on the design choices made throughout the formalisation, discusses alternatives
1600 that were explored, and identifies limitations of the current mechanisation.
1601

1602 *Evolution of the session fidelity proof.* The proof of session fidelity underwent significant revision
1603 before reaching its current form. An early iteration attempted to prove typability by induction on
1604 the typing derivation rather than on the process transition. This approach was poorly aligned with
1605 the theorem statement. In the parallel case, inverting a typing derivation exposes two independent
1606 sub-derivations, while the process transition determines which component actually steps. The
1607 proof therefore has no direct way to identify the sub-derivation that should be transformed and
1608 reassembled into the target typing derivation. The final proof instead proceeds by induction on
1609 `ProcStep_m` and inverts the source typing derivation in each case. This aligns the induction with the
1610 observed operational step and gives direct access to the typing rule compatible with the process
1611 shape.

1612 The treatment of environment transitions also changed during the development. An earlier ap-
1613 proach attempted to invert `EnvStep_m` directly, requiring a substantial collection of inversion lemmas,
1614 such as `EnvStep_inv_bot`, `EnvStep_inv_one`, `EnvStep_inv_one_bot`, and `EnvStep_inv_link`,
1615 together with auxiliary `HyperEnv.Perm.extract_*` lemmas for communication patterns under
1616 restriction. This infrastructure was necessary because the environment transition relation carried
1617

enough information to reconstruct hyperenvironment structure from a transition alone. The current formulation instead treats EnvStep_m as a lightweight projection of TypingStep_m . Resource evolution is still recorded, but well-formedness and reconstruction are recovered from the typed transition. This removes a layer of environment-step inversion infrastructure and concentrates the difficult reasoning in the typability proof.

Proof-engineering tradeoffs. Several premises in the typing and transition relations are included for proof convenience rather than logical necessity. The most prominent example is the post-state disjointness proof hd2 in the parallel constructors par_1 and par_2 of TypingStep_m , discussed in Section 4.2.1. This premise is derivable from the pre-state disjointness proof hd1 , the freshness condition requiring the names introduced by the label to be disjoint from the non-stepping component, and the structural property that transitions only introduce names recorded by their labels. Keeping hd2 explicit avoids reconstructing this derived fact in every inductive case, at the cost of slightly cluttering the inductive definition. This is representative of a broader proof-engineering tradeoff: some redundancy in the definitions can make individual proofs shorter and more robust, even when it makes the relations less minimal.

A second source of proof overhead is the mismatch between concrete freshness witnesses in ProcStep_m and co-finite in the typing rules. The binding constructors of ProcStep_m record the particular names introduced by a transition, whereas the corresponding typing rules are stated co-finitely. The simulation proofs must therefore show that the co-finite typing premise can be instantiated with the concrete names chosen by the process transition. This is handled by auxiliary *all fresh* lemmas, such as those for the tensor and parr prefix binders. Reformulating ProcStep_m co-finitely would align the process and typing relations more directly and could reduce the need for these freshness-bridging lemmas, although its effect on the erasure simulation proofs remains to be checked.

Structural congruence. The formalisation does not include a structural congruence relation for processes. In the paper presentation of πLL , structural congruence identifies processes modulo the commutative, associative, and unit laws of parallel composition, together with scope manipulation for restriction. The mechanisation currently recovers the typing-level counterpart of these laws through the monoidal laws for hyperenvironments and the exchange rules of the typing relation. However, it does not prove that process syntax itself is closed under structural congruence. For example, processes such as $P \mid \mathbf{0}$ and P are not automatically interchangeable; the corresponding simplification must instead be reflected manually at the level of hyperenvironments using the monoidal identity lemmas.

This omission matters because the operational semantics is defined over the concrete syntax tree of processes. In particular, synchronisation rules apply to components exposed by the syntactic structure of parallel composition. Since synchronisation labels are binary, the two processes that synchronise must be visible as sibling components in the syntax tree. For example, in a process of the form $P \mid (Q \mid R)$, a synchronisation between P and Q cannot be exposed by commutativity alone and would require associativity to rearrange the process into $(P \mid Q) \mid R$. A full structural congruence development would therefore require proving that typing and transitions are preserved under an independent congruence relation. An alternative would be to build selected symmetry and reassociation principles directly into the transition relation, for example to account for the orientation of restricted endpoints or the nesting of parallel composition. Whether this would simplify the session-fidelity proofs and later behavioural theory, or merely shift the proof burden elsewhere, remains unexplored.

Current limitations. The static formalisation covers the full π LL calculus, but the operational semantics and metatheory currently target the multiplicative fragment π MLL. Within π MLL, the remaining gap is the `res` case of `typability_subject_reductionm`. This case requires reconstructing the post-transition hyperenvironment after stepping the opened body of a restriction. In particular, the proof must show that hyperenvironment components not involved in the transition are preserved, up to permutation, so that the outer cut typing derivation can be rebuilt. The required block-preservation infrastructure is largely in place, but the remaining case analysis has not yet been completed.

Several results from the paper metatheory also remain unmechanised, including the behavioural theory of bisimilarity and congruence, the diamond property, serialisation, non-interference, and readiness. These are discussed as directions for future work in the following section.

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, and freshness lemmas. Many proof steps reduce to showing that two hyperenvironments are equivalent up to permutation, that their components remain disjoint, or that a chosen name is fresh for the relevant process and environment. These obligations currently require explicit sequences of rewrites and structural lemma applications.

A dedicated aesop rule set equipped with the hyperenvironment permutation, disjointness, name-set, and freshness lemmas could automate much of this framing work and significantly reduce proof size. This would not replace the main inversion arguments, such as the remaining `res` case, but it could remove much of the repetitive bookkeeping around them. Similarly, aligning `ProcStepm` more closely with the co-finite style of the typing rules could reduce the number of explicit freshness-bridging obligations

7 Future Work

The most immediate directions for future work concern completing the remaining session-fidelity proof obligation and extending the dynamic metatheory to the full π LL calculus.

Completing the multiplicative fragment. The immediate remaining proof obligation is the `res` case of `typability_subject_reductionm`. This case requires reconstructing the post-transition hyperenvironment after stepping the opened body of a restriction. The key missing ingredient is a block-preservation argument for `TypingStepm`, showing that hyperenvironment components not involved in a transition are preserved up to permutation. This should provide the infrastructure needed to rebuild the outer cut typing derivation after applying the induction hypothesis to the opened body.

Co-finite quantification in ProcStep. The current `ProcStepm` relation uses concrete freshness witnesses for binding constructors, while the typing rules use co-finite quantification. This asymmetry introduces proof overhead particularly in the typability theorem, where the specific concrete names chosen by `ProcStepm` must be reconciled with the universally quantified premises of the corresponding typing rules. Refactoring `ProcStepm` to use co-finite quantification for its binding constructors (`tensor`, `parr`, `one_bot`, `tensor_parr` and `res`) would align the process and typing relations more closely and could reduce the need for auxiliary *all fresh* lemmas. It may also make some explicit freshness and post-state disjointness premises derivable rather than primitive, although its effect on the erasure simulation proofs remains to be explored.

Extending to full π LL. Once the multiplicative fragment is complete, extending the formalisation to the full π LL calculus appears to require additional proof engineering rather than a new formal architecture. The static model is already defined for the full calculus, and many typing inversion lemmas for the additive, polymorphic, axiom, and exponential constructs are already available.

The main work is therefore expected to lie in extending $\text{typability_subject_reduction}_m$ and in proving the hyperenvironment extraction lemmas needed for the additional synchronisation cases, rather than in the erasure projections or the unit-like transition axioms. Some additional work would also be needed to extend the operational semantics with the transition constructors corresponding to the full set of rules from Appendix B.2 of [Montesi and Peressotti \[2021\]](#).

The remaining prefix and unit-like transition axioms from Appendix B.2 of [Montesi and Peressotti \[2021\]](#) are expected to follow the same proof pattern as the already mechanised one and bot cases. This includes the additive selections, server-use actions, polymorphic send and receive actions, disposal and duplication send actions, and the link actions. In each case, the proof should proceed by inverting the source typing derivation, constructing the corresponding typed transition, and reusing the existing local-closure and freshness infrastructure. The polymorphic cases additionally rely on the type-substitution and shifting lemmas developed for the locally nameless representation, while the link and axiom-cut cases require the name-substitution lemmas. The axiom-cut interaction may require special treatment, since it combines restriction with name substitution; one possible approach is to add an explicit axiom-under-restriction transition constructor, analogous to the existing one_bot and tensor_parr synchronisation constructors.

The remaining synchronisation cases, such as $!w$, $!c$, $!?$, $\exists V$, $\oplus_1 \&$, and $\oplus_2 \&$, follow the same general restriction-synchronisation pattern as the multiplicative communication cases. They require isolating the interacting components, stepping the opened body, and reconstructing the resulting hyperenvironment up to permutation. Once the extraction lemmas for the current res case are completed, these cases are expected to follow by analogous reasoning, with the relevant connectives and environments substituted for their multiplicative counterparts.

The duplication and disposal receive cases are more involved than the other server actions, since they generate a larger continuation process and manipulate dependencies and the typing environment. A prior revision of the formalisation, where server dependencies were stated implicitly, included the auxiliary constructions for building these processes and manipulating the corresponding dependencies and typing environments. Since the well-typedness of those constructions had already been established, parts of this earlier infrastructure may be reusable when extending the current dynamic metatheory.

Structural congruence. A further direction is to mechanise process-level structural congruence and prove that it preserves typing and transitions. This would address the artificial stuck states that arise from the strict binary-tree structure of the process syntax. In particular, commutativity and associativity of parallel composition are needed for the syn rule to apply to processes that are not immediate siblings in the syntax tree, as discussed in [Section 6](#). A structural congruence relation would internalise the commutative, associative, and unit laws of parallel composition, as well as scope manipulation for restriction, rather than requiring these rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Scope manipulation is especially delicate in the locally nameless setting, since moving a process under a restriction binder requires explicit De Bruijn index management. For example, a scope-extrusion principle of the form

$$(\nu(\bullet, \bullet)P) \mid Q \equiv \nu(\bullet, \bullet)(P \mid Q')$$

would require Q' to be obtained by shifting the bound channel indices of Q by 2, while leaving its free names unchanged, to account for the two additional channel binders introduced by the restriction. This would require both a channel-level shift operation on bound indices and a process-level traversal lifting that operation through the syntax.

The tradeoff is between modularity and local proof convenience. A full structural congruence relation would require preservation theorems showing that typing and transitions are stable under each congruence principle, but these results would then be reusable throughout the metatheory. A rule-oriented approach instead adds symmetric variants of selected rules, or selected symmetry and reassociation principles, directly to the transition relation. This could account for the congruences most relevant to stepping while preserving the syntax-directed structure of the system, which is useful for proof search in Lean. However, these additional constructors would still introduce extra cases in later proofs. Although unordered collections of processes could avoid some commutativity issues, representing process syntax quotient-wise would make syntax-directed inversion and induction substantially harder in Lean, mirroring the difficulties that led environments to be represented as lists rather than finite sets.

Behavioural metatheory. The paper establishes a range of further behavioural results that are not mechanised in the current formalisation. These include the theory of similarity, bisimilarity, and behavioural congruence, as well as the result connecting similarity with typability. The paper also proves the main parallelism properties of the calculus, namely the diamond property, serialisation, and non-interference. Finally, it establishes readiness, which for the full π LL calculus, requires the additional notion of potential, an induction metric that gives an upper bound on the number of internal transitions a process can perform. This also yields productivity: infinite executions of well-typed processes must involve infinitely many observable interactions [Montesi and Peressotti 2021].

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, name-set, and freshness lemmas. Building a dedicated aesop rule set equipped with these lemmas, together with a curated collection of simplification lemmas for normalising names and disjointness goals, could automate much of this framing work and significantly reduce proof size. This would however not replace the main inversion arguments, such as the remaining res case, but it could remove much of the repetitive bookkeeping around them. Similarly, aligning ProcStep_m more closely with the co-finite style of the typing rules could reduce the number of explicit freshness-bridging obligations.

8 Conclusion

This thesis set out to mechanise the syntax, semantics, and metatheory of π LL in Lean. The goal was not to extend the source calculus, but to validate and clarify an existing theoretical framework by making its binding structure, freshness assumptions, linear resource accounting, and operational metatheory explicit in a proof assistant.

The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, and typing judgements. Processes are represented using a locally nameless encoding, while type variables are represented using De Bruijn indices. This avoids explicit reasoning about α -equivalence while preserving readable free channel names in the mechanised syntax. The cost of this choice is the need for local-closure predicates, opening and closing operations, and co-finite reasoning principles.

The operational semantics is mechanised for the multiplicative fragment π MLL through three related transition systems: a typed transition system over typing derivations, a process transition system obtained by erasing typing resources, and an environment transition system describing the corresponding evolution of hyperenvironments. This mirrors the erasure structure of the source theory while adapting it to Lean's relational presentation of inductive transition systems.

The main metatheoretic development establishes the structure of the session-fidelity argument for the mechanised fragment. Typed derivation steps project to both process and environment

transitions, and process transitions from well-typed sources can be lifted back to typed derivation steps in the completed multiplicative cases. Subject reduction is then obtained by composing this lifting result with the environment projection. The remaining proof obligation is the res case of `typability_subject_reductionm`, where the proof must reconstruct the post-transition hyperenvironment after stepping under restriction.

The mechanisation also exposes several proof-engineering tradeoffs. Representing environments and hyperenvironments as lists makes structural induction and inversion feasible, but requires explicit permutation, disjointness, and freshness reasoning. Similarly, the mismatch between concrete freshness witnesses in `ProcStepm` and co-finite premises in the typing rules introduces auxiliary proof obligations. Finally, the absence of process-level structural congruence keeps the transition systems syntax-directed, but requires some rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Overall, the development provides a substantial mechanised account of the static structure of full π LL and the dynamic metatheory of its multiplicative fragment. It confirms that the central erasure and session-fidelity architecture of the source theory can be represented in Lean, while isolating the remaining proof infrastructure needed to complete the π MLL development and extend the mechanisation to the full calculus.

References

- Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (*POPL '08*). ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443
- Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.
- Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.
- Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33
- Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0
- Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/fall2023/entries/logic-linear/>
- Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4
- Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.
- Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π -calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818
- Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.

A Code Availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: <https://github.com/MikkelBrixNielsen/LeanPiLL>

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions are available in the repository history. These earlier versions are not part of the submitted artefact. The appendix

contains only selected excerpts of the development; the full collection of definitions, auxiliary lemmas, and proof scripts is available in the accompanying codebase.

B Lean File Structure

```
PiLL/
  Model/
    Environments/
    HyperEnvironments/
    Judgement/
    Processes/
    STypes/
    Base.lean
  Semantics/
    EnvStep/
    JudgementStep/
    ProcStep/
    Labels.lean
  SessionFidelity/
    ProcStep.lean
    EnvStep.lean
    Typability.lean
    SubjectReduction.lean
```

Listing 1. Top-level structure of the Lean development.

C Common Lean Notation

```
class HasShift (Subject : Type) where
  shift : Subject -> Nat -> Nat -> Subject

notation:max S:max " ↑ " d:max ", " c:max => HasShift.shift S d c
notation:max S:max " ↑ " c:max => HasShift.shift S 0 c
notation:max S:max "+" => HasShift.shift S 0 1

class HasSubst (Subject Replacement Target : Type) where
  subst : Subject -> Replacement -> Target -> Subject

notation:max S "{" R " //" T "}" => HasSubst.subst S R T

class HasOpen (Subject Replacement Target : Type) where
  open_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "(" T " | " R ")" => HasOpen.open_ S R T
notation:max S:max "(" R ")" => HasOpen.open_ S R 0

class HasClose (Subject Replacement Target : Type) where
  close_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "<<" T " | " R ">>" => HasClose.close_ S R T
notation:max S:max "<<" R ">>" => HasClose.close_ S R 0
```

```

1912 class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
1913   open_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
1914
1915 notation:max S:max "(" T " | " R1 " , " R2 ")" => HasOpenTwo.open_ S R1 R2 T
1916 notation:max S:max "(" R1 " , " R2 ")" => HasOpenTwo.open_ S R1 R2 0
1917
1918 class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
1919   close_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
1920
1921 notation:max S:max "<<" T " | " R1 " , " R2 ">>" => HasCloseTwo.close_ S R1 R2 T
1922 notation:max S:max "<<" R1 " , " R2 ">>" => HasCloseTwo.close_ S R1 R2 0
1923
1924 class HasBracket (Subject Content Result : Type) where
1925   brack : Subject -> Content -> Result
1926
1927 class HasParen (Subject Content Result : Type) where
1928   paren : Subject -> Content -> Result
1929
1930 notation:80 x"[]" => HasBracket.brack x ()
1931 notation:80 x"[]" => HasBracket.brack x y
1932 notation:80 x"()" => HasParen.paren x ()
1933 notation:80 x"()" => HasParen.paren x y
1934
1935 class HasPerm ( $\alpha$  : Type) where
1936   perm :  $\alpha$  ->  $\alpha$  -> Prop
1937
1938 infixr:54 " ~ " => HasPerm.perm

```

Listing 2. Common typeclasses and notation used throughout the Lean development. These abbreviations are used uniformly for shifting, substitution, opening, closing, prefix notation, and permutation.

D Selected Core Definitions

D.1 Session Types

```

1944 abbrev Atom := Nat
1945 abbrev FTVar := Nat
1946 abbrev BTVar := Nat
1947
1948 inductive TVar : Type
1949   | free   (x : FTVar)
1950   | bound  (x : BTVar)
1951 deriving DecidableEq, Repr
1952
1953 inductive Types : Type where
1954   | atom      (a : Atom)
1955   | atomDual  (a : Atom)
1956   | var       (v : TVar)
1957   | varDual   (v : TVar)
1958   | one
1959   | bot
1960   | tensor    (A B : Types)

```

```

1961 | parr      (A B : Types)
1962 | oplus     (A B : Types)
1963 | amp       (A B : Types)
1964 | bang      (A : Types)
1965 | quest     (A : Types)
1966 | forall_   (A : Types)
1967 | exists_   (A : Types)
1968 deriving DecidableEq, BEq, Repr

```

Listing 3. Locally nameless representation of type variables and the Lean definition of session types. Free type variables are represented explicitly, while bound type variables are represented by De Bruijn indices.

```

1972 def Types.dual : Types -> Types
1973 | .atom a      => .atomDual a
1974 | .atomDual a  => .atom a
1975 | .var v       => .varDual v
1976 | .varDual v   => .var v
1977 | .one         => .bot
1978 | .bot        => .one
1979 | .tensor A B  => .parr (dual A) (dual B)
1980 | .parr A B    => .tensor (dual A) (dual B)
1981 | .oplus A B   => .amp (dual A) (dual B)
1982 | .amp A B     => .oplus (dual A) (dual B)
1983 | .bang A      => .quest (dual A)
1984 | .quest A     => .bang (dual A)
1985 | .forall_ A   => .exists_ (dual A)
1986 | .exists_ A   => .forall_ (dual A)

```

Listing 4. Duality on session types. The definition exchange each connective with its dual.

```

1989 def TVar.lc : Nat -> TVar -> Prop
1990 | _, .free _  => True
1991 | k, .bound i => i < k
1992
1993 def Types.lc : Nat -> Types -> Prop
1994 | _, .atom _  => True
1995 | _, .atomDual _ => True
1996 | k, .var v   => TVar.lc k v
1997 | k, .varDual v => TVar.lc k v
1998 | _, .one     => True
1999 | _, .bot     => True
2000 | k, .tensor A B => A.lc k ∧ B.lc k
2001 | k, .parr A B  => A.lc k ∧ B.lc k
2002 | k, .oplus A B => A.lc k ∧ B.lc k
2003 | k, .amp A B   => A.lc k ∧ B.lc k
2004 | k, .bang A    => A.lc k
2005 | k, .quest A   => A.lc k
2006 | k, .forall_ A => A.lc (k+1)
2007 | k, .exists_ A => A.lc (k+1)
2008
2009 def Types.lc_0 : Types -> Prop := Types.lc 0

```

Listing 5. Local closure predicates for type variables and session types. A bound type variable is locally closed at depth k when its De Bruijn index is smaller than k . Quantifiers increase the depth by one.

```

def TVar.shift (d c : Nat) : TVar -> TVar
| .bound i => if i < d then .bound i else .bound (i + c)
| .free n => .free n

instance : HasShift TVar where
  shift v d c := TVar.shift d c v

def Types.shift (d c : Nat) : Types -> Types
| .var v      => .var (v.shift d c)
| .varDual v  => .varDual (v.shift d c)
| .forall_ A  => .forall_ (A.shift (d + 1) c)
| .exists_ A  => .exists_ (A.shift (d + 1) c)
| .tensor A B => .tensor (A.shift d c) (B.shift d c)
| .parr A B   => .parr (A.shift d c) (B.shift d c)
| .quest A    => .quest (A.shift d c)
| .bang A     => .bang (A.shift d c)
| .amp A B    => .amp (A.shift d c) (B.shift d c)
| .oplus A B  => .oplus (A.shift d c) (B.shift d c)
| t           => t

instance : HasShift Types
where shift T d c := Types.shift

```

Listing 6. Shifting for type variables and session types. The parameter d is the cutoff depth and c is the shift amount. Bound variables below the cutoff are locally bound and left unchanged; variables at or above the cutoff are shifted.

```

def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.bound i) =>
    if i == k then A
    else if i > k then .var (.bound (i - 1))
    else .var (.bound i)
  | .varDual (.bound i) =>
    if i == k then (.dual A)
    else if i > k then .varDual (.bound (i - 1))
    else .varDual (.bound i)
  | .var (.free i)    => .var (.free i)
  | .varDual (.free i) => .varDual (.free i)
  | .forall_ B        => .forall_ (B.subst (shift 0 1 A) (k + 1))
  | .exists_ B        => .exists_ (B.subst (shift 0 1 A) (k + 1))
  | .tensor L R       => .tensor (L.subst A k) (R.subst A k)
  | .parr L R         => .parr (L.subst A k) (R.subst A k)
  | .quest B          => .quest (B.subst A k)
  | .bang B           => .bang (B.subst A k)
  | .amp L R          => .amp (L.subst A k) (R.subst A k)

```

```

2059 | .oplus L R      => .oplus (L.subst A k) (R.subst A k)
2060 | t              => t
2061
2062 instance : HasSubst Types Types Nat
2063   where subst := Types.subst

```

Listing 7. Substitution for type variables. Substitution replaces the bound variable at index k with the type A , shifts A when traversing under quantifiers, and adjusts indices above k to account for the removed binder.

D.2 Processes

```

2070 abbrev FPName := Nat
2071 abbrev BPNName := Nat
2072
2073 inductive Channel : Type where
2074   | free    (x : FPName)
2075   | bound   (x : BPNName)
2076 deriving DecidableEq, BEq, Repr
2077
2078 prefix:max "#" => Channel.free
2079 prefix:max "$" => Channel.bound
2080
2081 inductive Proc : Type where
2082   | nil
2083   | one      (x : Channel) (P : Proc)
2084   | bot      (x : Channel) (P : Proc)
2085   | tensor   (x : Channel) (P : Proc)
2086   | parr     (x : Channel) (P : Proc)
2087   | cut      (P : Proc)
2088   | par      (P Q : Proc)
2089   | selectL  (x : Channel) (P : Proc)
2090   | selectR  (x : Channel) (P : Proc)
2091   | amp      (x : Channel) (P Q : Proc)
2092   | output   (x : Channel) (P : Proc) (A : Types)
2093   | input    (x : Channel) (P : Proc)
2094   | server   (x : Channel) (zs : Finset Channel) (P : Proc)
2095   | consume  (x : Channel) (P : Proc)
2096   | duplicate (x : Channel) (P : Proc)
2097   | dispose  (x : Channel) (P : Proc)
2098   | link     (x y : Channel)
2099 deriving DecidableEq, BEq

```

Listing 8. Locally nameless representation of channels and process syntax. Free channels are represented explicitly, while bound channels are represented by De Bruijn indices.

```

2102 def Channel.open (u v : Channel) (k : Nat) : Channel :=
2103   match u with
2104   | bound i => if i == k then v else bound i
2105   | c      => c

```

```

2108 instance : HasOpen Channel Channel Nat where
2109   open_ u v k := Channel.open u v k
2110
2111 def Finset.open (zs : Finset Channel) (v : Channel) (k : Nat) : Finset Channel :=
2112   zs.image (fun u => u.open v k)
2113
2114 instance : HasOpen (Finset Channel) Channel Nat where
2115   open_ zs v k := Finset.open zs v k
2116
2117 def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
2118   match P with
2119   | .nil                => .nil
2120   | .one x P            => .one (x.open u k) (P.open u k)
2121   | .bot x P            => .bot (x.open u k) (P.open u k)
2122   | .tensor x P         => .tensor (x.open u k) (P.open u (k + 1))
2123   | .parr x P           => .parr (x.open u k) (P.open u (k + 1))
2124   | .cut P              => .cut (P.open u (k + 2))
2125   | .par P Q            => .par (P.open u k) (Q.open u k)
2126   | .server x zs P      => .server (x.open u k) (zs.open u k) (P.open u k)
2127   | .duplicate x P      => .duplicate (x.open u k) (P.open u (k + 1))
2128   | .consume x P        => .consume (x.open u k) (P.open u k)
2129   | .dispose x P        => .dispose (x.open u k) (P.open u k)
2130   | .selectL x P        => .selectL (x.open u k) (P.open u k)
2131   | .selectR x P        => .selectR (x.open u k) (P.open u k)
2132   | .amp x P Q          => .amp (x.open u k) (P.open u k) (Q.open u k)
2133   | .output x P A       => .output (x.open u k) (P.open u k) A
2134   | .input x P          => .input (x.open u k) (P.open u k)
2135   | .link x y           => .link (x.open u k) (y.open u k)
2136
2137 instance : HasOpen Proc Channel Nat where
2138   open_ P v k := Proc.open P v k
2139
2140 instance : HasOpenTwo Proc Channel Channel Nat where
2141   open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)

```

Listing 9. Opening for channels and processes. Opening replaces the bound index at the current depth with a concrete channel. Constructors that bind channels increase the depth before traversing their continuation.

```

2144 def Channel.close (u : Channel) (x : FPNat) (k : Nat) : Channel :=
2145   match u with
2146   | .free name => if x == name then .bound k else .free name
2147   | .bound i  => .bound i
2148
2149 instance : HasClose Channel FPNat Nat where
2150   close_ u x k := Channel.close u x k
2151
2152 def Finset.close (zs : Finset Channel) (x : FPNat) (k : Nat) : Finset Channel :=
2153   zs.image (fun u => u.close x k)
2154
2155 instance : HasClose (Finset Channel) FPNat Nat where
2156   close_ zs x k := Finset.close zs x k

```

```

2157
2158 def Proc.close (P : Proc) (z : FPNName) (k : Nat) : Proc :=
2159   match P with
2160   | .nil                => .nil
2161   | .one x P            => .one (x.close z k) (P.close z k)
2162   | .bot x P            => .bot (x.close z k) (P.close z k)
2163   | .tensor x P         => .tensor (x.close z k) (P.close z (k + 1))
2164   | .parr x P           => .parr (x.close z k) (P.close z (k + 1))
2165   | .cut P              => .cut (P.close z (k + 2))
2166   | .par P Q            => .par (P.close z k) (Q.close z k)
2167   | .selectL x P        => .selectL (x.close z k) (P.close z k)
2168   | .selectR x P        => .selectR (x.close z k) (P.close z k)
2169   | .amp x P Q          => .amp (x.close z k) (P.close z k) (Q.close z k)
2170   | .output x P A       => .output (x.close z k) (P.close z k) A
2171   | .input x P          => .input (x.close z k) (P.close z k)
2172   | .server x zs P      => .server (x.close z k) (zs.close z k) (P.close z k)
2173   | .consume x P        => .consume (x.close z k) (P.close z k)
2174   | .duplicate x P      => .duplicate (x.close z k) (P.close z (k + 1))
2175   | .dispose x P        => .dispose (x.close z k) (P.close z k)
2176   | .link x y           => .link (x.close z k) (y.close z k)
2177
2178 instance : HasClose Proc FPNName Nat where
2179   close_ P x k := Proc.close P x k
2180
2181 instance : HasCloseTwo Proc FPNName FPNName Nat where
2182   close_ P x y k := (Proc.close (Proc.close P y (k + 1)) x k)

```

Listing 10. Closing for channels and processes. Closing abstracts a free channel name into a bound index at the current depth. Multi-name binders are handled by increasing the depth under the binder.

```

2185
2186 def Channel.lc : Nat -> Channel -> Prop
2187 | _, .free _    => True
2188 | k, .bound i   => i < k
2189
2190 def Finset.lc (zs : Finset Channel) (k : Nat) : Prop :=
2191   ∀ z ∈ zs, z.lc k
2192
2193 def Proc.lc : Nat -> Nat -> Proc -> Prop
2194 | _, _, .nil                => True
2195 | k, n, .one x P            => x.lc k ∧ P.lc k n
2196 | k, n, .bot x P            => x.lc k ∧ P.lc k n
2197 | k, n, .tensor x P         => x.lc k ∧ P.lc (k + 1) n
2198 | k, n, .parr x P           => x.lc k ∧ P.lc (k + 1) n
2199 | k, n, .cut P              => P.lc (k + 2) n
2200 | k, n, .par P Q            => P.lc k n ∧ Q.lc k n
2201 | k, n, .selectL x P        => x.lc k ∧ P.lc k n
2202 | k, n, .selectR x P        => x.lc k ∧ P.lc k n
2203 | k, n, .amp x P Q          => x.lc k ∧ P.lc k n ∧ Q.lc k n
2204 | k, n, .output x P A       => x.lc k ∧ P.lc k n ∧ A.lc n
2205 | k, n, .input x P          => x.lc k ∧ P.lc k (n + 1)
2206 | k, n, .server x zs P      => x.lc k ∧ zs.lc k ∧ P.lc k n

```



```

2206 | k, n, .consume x P    => x.lc k ∧ P.lc k n
2207 | k, n, .duplicate x P  => x.lc k ∧ P.lc (k + 1) n
2208 | k, n, .dispose x P    => x.lc k ∧ P.lc k n
2209 | k, _, .link x y       => x.lc k ∧ y.lc k
2210
2211 def Proc.lc_0 : Proc -> Prop := Proc.lc 0 0

```

Listing 11. Local closure for channels and processes. The first index tracks bound channel depth, while the second tracks bound type-variable depth. Channel binders increase the channel depth, while type input increases the type-variable depth.

```

2216
2217 def Channel.f : Channel -> Finset FPNName
2218 | .free x    => {x}
2219 | .bound _   => {}
2220
2221 def Finset.f (zs : Finset Channel) : Finset FPNName :=
2222   zs.biUnion Channel.f
2223
2224 def Proc.f : Proc -> Finset FPNName
2225 | .tensor x P    => x.f ∪ P.f
2226 | .parr x P      => x.f ∪ P.f
2227 | .one x P       => x.f ∪ P.f
2228 | .bot x P       => x.f ∪ P.f
2229 | .cut P         => P.f
2230 | .par P Q       => P.f ∪ Q.f
2231 | .nil          => {}
2232 | .selectL x P   => x.f ∪ P.f
2233 | .selectR x P   => x.f ∪ P.f
2234 | .amp x P Q     => x.f ∪ P.f ∪ Q.f
2235 | .output x P _  => x.f ∪ P.f
2236 | .input x P     => x.f ∪ P.f
2237 | .server x zs P => x.f ∪ zs.f ∪ P.f
2238 | .consume x P   => x.f ∪ P.f
2239 | .duplicate x P => x.f ∪ P.f
2240 | .dispose x P   => x.f ∪ P.f
2241 | .link x y      => x.f ∪ y.f

```

Listing 12. Free-name computation for channels, finite sets of channels, and processes. Bound indices contribute no free names, while free channel occurrences are collected structurally through the process syntax.

```

2244 def FPNName.subst (R T : FPNName) : FPNName -> FPNName :=
2245   (fun x => if x = T then R else x)
2246
2247 instance : HasSubst FPNName FPNName FPNName where
2248   subst x R T := FPNName.subst R T x
2249
2250 def Channel.subst (R T : FPNName) : Channel -> Channel
2251 | .free x => if x == T then .free R else .free x
2252 | .bound i => .bound i
2253
2254 instance : HasSubst Channel FPNName FPNName where

```

```

2255   subst C R T := Channel.subst R T C
2256
2257   def Finset.subst (zs : Finset Channel) (R T : FPName) : Finset Channel :=
2258     zs.image (fun u => u.subst R T)
2259
2260   instance : HasSubst (Finset Channel) FPName FPName where
2261     subst S R T := Finset.subst S R T
2262
2263   def Proc.substNames (R T : FPName) : Proc -> Proc
2264   | .nil           => .nil
2265   | .one x P       => .one (x.subst R T) (P.substNames R T)
2266   | .bot x P       => .bot (x.subst R T) (P.substNames R T)
2267   | .tensor x P    => .tensor (x.subst R T) (P.substNames R T)
2268   | .parr x P      => .parr (x.subst R T) (P.substNames R T)
2269   | .cut P         => .cut (P.substNames R T)
2270   | .par P Q       => .par (P.substNames R T) (Q.substNames R T)
2271   | .selectL x P   => .selectL (x.subst R T) (P.substNames R T)
2272   | .selectR x P   => .selectR (x.subst R T) (P.substNames R T)
2273   | .amp x P Q     => .amp (x.subst R T) (P.substNames R T) (Q.substNames R T)
2274   | .output x P A  => .output (x.subst R T) (P.substNames R T) A
2275   | .input x P     => .input (x.subst R T) (P.substNames R T)
2276   | .server x zs P => .server (x.subst R T) (zs.subst R T) (P.substNames R T)
2277   | .consume x P   => .consume (x.subst R T) (P.substNames R T)
2278   | .duplicate x P => .duplicate (x.subst R T) (P.substNames R T)
2279   | .dispose x P   => .dispose (x.subst R T) (P.substNames R T)
2280   | .link x y      => .link (x.subst R T) (y.subst R T)
2281
2282   instance : HasSubst Proc FPName FPName where
2283     subst P R T := Proc.substNames R T P

```

Listing 13. Substitution of free channel names in processes. The substitution acts only on free channels and leaves bound indices unchanged.

```

2285   def Proc.substTypes (A : Types) (k : Nat) : Proc -> Proc
2286   | .nil           => .nil
2287   | .one x P       => .one x (P.substTypes A k)
2288   | .bot x P       => .bot x (P.substTypes A k)
2289   | .tensor x P    => .tensor x (P.substTypes A k)
2290   | .parr x P      => .parr x (P.substTypes A k)
2291   | .cut P         => .cut (P.substTypes A k)
2292   | .par P Q       => .par (P.substTypes A k) (Q.substTypes A k)
2293   | .selectL x P   => .selectL x (P.substTypes A k)
2294   | .selectR x P   => .selectR x (P.substTypes A k)
2295   | .amp x P Q     => .amp x (P.substTypes A k) (Q.substTypes A k)
2296   | .output x P B  => .output x (P.substTypes A k) (B.subst A k)
2297   | .input x P     => .input x (P.substTypes (A.shift 0 1) (k + 1))
2298   | .server x zs P => .server x zs (P.substTypes A k)
2299   | .consume x P   => .consume x (P.substTypes A k)
2300   | .duplicate x P => .duplicate x (P.substTypes A k)
2301   | .dispose x P   => .dispose x (P.substTypes A k)
2302   | .link x y      => .link x y
2303

```

```

2304 instance : HasSubst Proc Types Nat where
2305   subst P A k := Proc.substTypes A k P
2306
2307 def Proc.shiftTypes (d c : Nat) : Proc -> Proc
2308 | .nil           => .nil
2309 | .one x P       => .one x (P.shiftTypes d c)
2310 | .bot x P       => .bot x (P.shiftTypes d c)
2311 | .tensor x P    => .tensor x (P.shiftTypes d c)
2312 | .parr x P      => .parr x (P.shiftTypes d c)
2313 | .cut P         => .cut (P.shiftTypes d c)
2314 | .par P Q       => .par (P.shiftTypes d c) (Q.shiftTypes d c)
2315 | .selectL x P   => .selectL x (P.shiftTypes d c)
2316 | .selectR x P   => .selectR x (P.shiftTypes d c)
2317 | .amp x P Q     => .amp x (P.shiftTypes d c) (Q.shiftTypes d c)
2318 | .output x P A  => .output x (P.shiftTypes d c) (A.shift d c)
2319 | .input x P     => .input x (P.shiftTypes (d + 1) c)
2320 | .server x zs P => .server x zs (P.shiftTypes d c)
2321 | .consume x P   => .consume x (P.shiftTypes d c)
2322 | .duplicate x P => .duplicate x (P.shiftTypes d c)
2323 | .link x y      => .link x y
2324
2325 instance : HasShift Proc where
2326   shift P d c := Proc.shiftTypes d c P

```

Listing 14. Type substitution and type-variable shifting lifted to processes. Type substitution only affects type annotations and traverses under type input by shifting the substituted type.

```

2330 def freshName (s : Finset FPNName) : FPNName :=
2331   s.sup id + 1
2332
2333 lemma fresh_is_fresh (s : Finset FPNName) (x : FPNName) (h : x ∈ s) :
2334   id x < freshName s
2335
2336 lemma exists_one_fresh (L : Finset FPNName) :
2337   ∃ u, u ∉ L
2338
2339 lemma exists_two_fresh (L : Finset FPNName) :
2340   ∃ u v, u ∉ L ∧ v ∉ L ∧ u ≠ v

```

Listing 15. Fresh-name generation for finite sets of free process names.

The construction uses the fact that free process names are represented as natural numbers. For a finite set L , $\text{freshName } L$ is defined as one more than the maximum element of L . The lemma fresh_is_fresh states that every element of L is strictly smaller than this value, so $\text{freshName } L$ cannot itself be a member of L . The one-name freshness lemma chooses this value directly. The two-name version first chooses a fresh name u for L , then chooses a second name v fresh for $L \cup \{u\}$, ensuring both $v \notin L$ and $u \neq v$.

D.3 Environments

```

2351 abbrev Elem := (FPNName × Types)

```

```

2353 abbrev Elem.mk (x : Fpname) (A : Types) : Elem := (x, A)
2354 infixr:68 " : " => Elem.mk
2355
2356 abbrev Env := List Elem
2357
2358 def Env.names (Γ : Env) : Finset Fpname :=
2359   (Γ.map Prod.fst).toFinset
2360
2361 abbrev Env.merge (Γ ~ Δ : Env) : Env := Γ ++ Δ
2362 infixl:69 ", " => Env.merge
2363
2364 def Env.Nodup (Γ : Env) : Prop :=
2365   (Γ.map Prod.fst).Nodup
2366
2367 @[simp] def Env.disjoint (Γ Δ : Env) : Prop :=
2368   Disjoint Γ.names Δ.names

```

Listing 16. Lean representation of typing environments. Environments are implemented as lists of name/type pairs, with names extracted into finite sets for freshness and disjointness reasoning.

```

2374 def Env.lc (k : Nat) (Γ : Env) : Prop :=
2375   ∀ (x, A), (x, A) ∈ Γ, A.lc k
2376
2377 def Env.serverUsable (Γ : Env) : Prop :=
2378   ∀ p, p ∈ Γ -> (p.snd).isServerUsable
2379
2380 prefix:max "?_e" => Env.serverUsable
2381
2382 def Env.freeTypes (Γ : Env) :=
2383   Γ.foldl (fun acc (_, A) => acc ∪ A.freeTypes) ∅
2384
2385 notation "ft(" Γ ")"_e => Env.freeTypes Γ

```

Listing 17. Selected structural predicates on environments. Local closure lifts type local closure pointwise to every type in the environment, while server usability is used by the exponential rules.

```

2391 def Env.shiftTypes (d c : Nat) (Γ : Env) : Env :=
2392   Γ.map (fun (x, A) => (x, A.shift d c))
2393
2394 instance : HasShift Env where
2395   shift Γ d c := Env.shiftTypes d c Γ
2396
2397 def Env.substTypes (Γ : Env) (A : Types) (k : Nat) : Env :=
2398   Γ.map (fun (x, B) => (x, B.subst A k))
2399
2400 instance : HasSubst Env Types Nat where
2401   subst Γ A k := Env.substTypes Γ A k

```

Listing 18. Type-level operations lifted pointwise to environments. These operations are used by the polymorphic typing rules and by substitution-preservation lemmas.

```
def Env.substNames (Γ : Env) (R T : FName) : Env :=
  Γ.map (fun (x, A) => if x == T then (R, A) else (x, A))

instance : HasSubst Env FName FName where
  subst Γ R T := Env.substNames Γ R T
```

Listing 19. Substitution of free process names in environments. Since environments map free channel names to types, name substitution acts on the first component of each environment entry.

```
@[simp] lemma Env.names_nil :
  Env.names [] = ∅ := by
  ...

@[simp] lemma Env.names_singleton {x : FName} {A : Types} :
  Env.names [x :: A] = {x} := by
  ...

@[simp] lemma Env.names_distributes {Γ : Env} {x : FName} {A : Types} :
  Env.names (x : A :: Γ) = {x} ∪ Γ.names := by
  ...

@[simp] lemma Env.names_merge {Γ Δ : Env} :
  (Γ, Δ).names = Γ.names ∪ Δ.names := by
  ...
```

Listing 20. Selected name-set simplification lemmas for environments. These lemmas normalise environment names through the list representation and are used repeatedly in freshness and disjointness proofs.

The proofs of these lemmas are mostly structural consequences of the list representation. Name-set lemmas unfold `Env.names` and reduce to facts about `List.map` and `Finset.toFinset`. Permutation lemmas use the fact that list permutation preserves membership and therefore preserves the induced name set. The full proof scripts are included in the accompanying Lean development.

```
lemma Env.names_eq_of_perm {Γ Δ : Env} (h : Γ ~ Δ) :
  Γ.names = Δ.names := by
  ...

lemma Env.Nodup_perm {Γ Δ : Env} :
  Γ ~ Δ -> Γ.Nodup -> Δ.Nodup := by
  ...

lemma Env.perm_disjoint {Γ Δ Ξ : Env} (hP : Γ ~ Δ) :
  Γ.disjoint Ξ ↔ Δ.disjoint Ξ := by
  ...

lemma Env.disjoint_cons_iff {Γ Δ : Env} {x y : FName} {A : Types} :
```

```

Disjoint (Env.names (x ::= A ::  $\Gamma$ ))
  (Env.names (y ::=  $A^\perp$  ::  $\Delta$ ))  $\leftrightarrow$ 
(y  $\neq$  x  $\wedge$  y  $\notin$   $\Gamma$ .names  $\wedge$ 
 x  $\notin$   $\Delta$ .names  $\wedge$  Disjoint  $\Gamma$ .names  $\Delta$ .names) := by
...
```

Listing 21. Selected permutation and disjointness lemmas for environments. These lemmas recover order-insensitive reasoning over the list representation of environments.

The lemmas above connect the list-based implementation of environments with the order-insensitive reasoning expected of typing environments. Permuting an environment preserves its set of channel names, and therefore also preserves disjointness against any third environment. The disjointness expansion lemma rewrites disjointness between two extended environments into explicit freshness obligations on the head names and the remaining environments. These lemmas are repeatedly used when reconstructing typing derivations after exchange or when extracting the freshness conditions needed by transition rules.

```

lemma Env.merge_unitL ( $\Gamma$  : Env) :
  ([ ] : Env),  $\Gamma$  =  $\Gamma$  := by
...

lemma Env.merge_unitR ( $\Gamma$  : Env) :
   $\Gamma$ , ([ ] : Env) =  $\Gamma$  := by
...

lemma Env.merge_assoc ( $\Gamma$   $\Delta$   $\Xi$  : Env) :
   $\Gamma$ ,  $\Delta$ ,  $\Xi$  =  $\Gamma$ , ( $\Delta$ ,  $\Xi$ ) := by
...

lemma Env.merge_comm ( $\Gamma$   $\Delta$  : Env) :
   $\Gamma$ ,  $\Delta$   $\sim$   $\Delta$ ,  $\Gamma$  := by
...
```

Listing 22. Selected monoidal laws for environment merge. Since merge is implemented as list append, unit and associativity hold directly, while commutativity is recovered up to permutation.

The merge operation on environments is implemented as list append. Consequently, the empty environment acts as a left and right unit, and associativity follows directly from associativity of append. Commutativity, however, cannot hold as definitional equality because lists remember order. Instead, the formalisation proves commutativity up to `List.Perm`. These lemmas recover the intended order-insensitive behaviour of typing environments while retaining a list-based representation that remains convenient for induction and inversion.

D.4 Hyperenvironments

```

abbrev HyperEnv := List Env

notation "["  $\Gamma$  "]" => ([ $\Gamma$ ] : HyperEnv)

def HyperEnv.names ( $\mathcal{G}$  : HyperEnv) : Finset FPNName :=
   $\mathcal{G}$ .foldr (fun  $\Gamma$  acc =>  $\Gamma$ .names  $\cup$  acc)  $\emptyset$ 
```

```

2500 def HyperEnv.Nodup ( $\mathcal{G}$  : HyperEnv) : Prop :=
2501    $\forall \Gamma \in \mathcal{G}, \text{Env.Nodup } \Gamma$ 
2502
2503 def HyperEnv.PairwiseDisjoint ( $\mathcal{G}$  : HyperEnv) : Prop :=
2504   List.Pairwise Env.disjoint  $\mathcal{G}$ 
2505
2506 @[simp] def HyperEnv.disjoint ( $\mathcal{G} \ \mathcal{H}$  : HyperEnv) : Prop :=
2507   Disjoint  $\mathcal{G}.\text{names}$   $\mathcal{H}.\text{names}$ 
2508

```

Listing 23. Lean representation of hyperenvironments. A hyperenvironment is implemented as a list of environments, corresponding to the separated components of a typing context.

The definitions above lift the environment-level resource invariants to hyperenvironments. The function `HyperEnv.names` collects all channel names appearing in any component, while `HyperEnv.Nodup` requires each individual environment to contain no duplicate names. The stronger invariant `HyperEnv.PairwiseDisjoint` states that distinct components are pairwise disjoint, capturing the linear separation of resources across parallel components.

```

2509 inductive HyperEnv.Perm : HyperEnv -> HyperEnv -> Prop where
2510   | nil :
2511     Perm [] []
2512
2513   | cons { $\Gamma \ \Delta$  : Env} { $\mathcal{G} \ \mathcal{H}$  : HyperEnv} :
2514      $\Gamma \sim \Delta \rightarrow \text{Perm } \mathcal{G} \ \mathcal{H} \rightarrow \text{Perm } (\Gamma :: \mathcal{G}) (\Delta :: \mathcal{H})$ 
2515
2516   | swap ( $\Gamma \ \Delta$  : Env) ( $\mathcal{G}$  : HyperEnv) :
2517     Perm ( $\Gamma :: \Delta :: \mathcal{G}$ ) ( $\Delta :: \Gamma :: \mathcal{G}$ )
2518
2519   | trans { $\mathcal{G} \ \mathcal{H} \ \mathcal{J}$  : HyperEnv} :
2520     Perm  $\mathcal{G} \ \mathcal{H} \rightarrow \text{Perm } \mathcal{H} \ \mathcal{J} \rightarrow \text{Perm } \mathcal{G} \ \mathcal{J}$ 
2521
2522 instance : HasPerm HyperEnv where
2523   perm := HyperEnv.Perm
2524

```

Listing 24. Deep permutation of hyperenvironments. Hyperenvironment permutation allows both reordering of environment components and permutation inside each individual environment.

The permutation relation in above listing is a deep permutation relation. It does not only reorder the outer list of environments; the `cons` constructor also allows the head environments themselves to be permuted. This is essential because both environments and hyperenvironments are implemented as lists, while the typing theory treats exchange as structurally irrelevant.

```

2525 def HyperEnv.shiftTypes (d c : Nat) ( $\mathcal{G}$  : HyperEnv) : HyperEnv :=
2526    $\mathcal{G}.\text{map } (\text{fun } \Gamma \Rightarrow \Gamma.\text{shiftTypes } d \ c)$ 
2527
2528 instance : HasShift HyperEnv where
2529   shift  $\mathcal{G} \ d \ c := \text{HyperEnv.shiftTypes } d \ c \ \mathcal{G}$ 
2530
2531 def HyperEnv.substTypes ( $\mathcal{G}$  : HyperEnv) (A : Types) (k : Nat) : HyperEnv :=
2532    $\mathcal{G}.\text{map } (\text{fun } \Gamma \Rightarrow \Gamma.\text{substTypes } A \ k)$ 
2533

```



```

instance : HasSubst HyperEnv Types Nat where
  subst  $\mathcal{G}$  A k := HyperEnv.substTypes  $\mathcal{G}$  A k

def HyperEnv.substNames ( $\mathcal{G}$  : HyperEnv) (R T : FPNName) : HyperEnv :=
   $\mathcal{G}$ .map (fun  $\Gamma$  =>  $\Gamma$ .substNames R T)

instance : HasSubst HyperEnv FPNName FPNName where
  subst  $\mathcal{G}$  R T := HyperEnv.substNames  $\mathcal{G}$  R T

```

Listing**25. Type-**

and

name-level operations lifted to hyperenvironments. These definitions lift the corresponding environment operations componentwise.

The operations in the above listing are lifted pointwise from environments to hyperenvironments. This reflects the implementation choice that a hyperenvironment is a list of independent environment components: type shifting, type substitution, and name substitution do not change the hyperenvironment structure, but only transform each component. The corresponding typeclass instances allow the same notation for shifting and substitution to be reused uniformly across types, processes, environments, and hyperenvironments.

```

@[simp] lemma HyperEnv.names_nil :
  HyperEnv.names [] =  $\emptyset$  := by
  ...

@[simp] lemma HyperEnv.names_singleton { $\Gamma$  : Env} :
  HyperEnv.names [ $\Gamma$ ] =  $\Gamma$ .names := by
  ...

@[simp] lemma HyperEnv.names_cons { $\Gamma$  : Env} { $\mathcal{G}$  : HyperEnv} :
  HyperEnv.names ( $\Gamma :: \mathcal{G}$ ) =  $\Gamma$ .names  $\cup$   $\mathcal{G}$ .names := by
  ...

lemma HyperEnv.subset_names_of_mem { $\Gamma$  : Env} { $\mathcal{G}$  : HyperEnv}
  (h :  $\Gamma \in \mathcal{G}$ ) :
   $\Gamma$ .names  $\subset$   $\mathcal{G}$ .names := by
  ...

lemma HyperEnv.names_eq_of_perm { $\mathcal{G}$   $\mathcal{H}$  : HyperEnv}
  (hP :  $\mathcal{G} \sim \mathcal{H}$ ) :
   $\mathcal{G}$ .names =  $\mathcal{H}$ .names := by
  ...

```

Listing 26. Selected simplification lemmas for hyperenvironment names. These lemmas normalise name-set goals involving empty, singleton, cons, and merged hyperenvironments.

The lemmas in the above listing connect the list implementation of hyperenvironments with finite-set reasoning about names. They are used to reduce freshness and disjointness side conditions to ordinary finite-set goals. In particular, `HyperEnv.names_eq_of_perm` shows that deep permutation preserves the set of names appearing in a hyperenvironment.

```

abbrev HyperEnv.merge ( $\mathcal{G}$   $\mathcal{H}$  : HyperEnv) : HyperEnv :=  $\mathcal{G} ++ \mathcal{H}$ 

infixl:65 "  $\mid_h$  " => HyperEnv.merge

```

```

2598 lemma HyperEnv.merge_unitL ( $\mathcal{G}$  : HyperEnv) :
2599    $\emptyset \mid_h \mathcal{G} = \mathcal{G} := \text{by}$ 
2600   ...
2601
2602 lemma HyperEnv.merge_unitR ( $\mathcal{G}$  : HyperEnv) :
2603    $\mathcal{G} \mid_h \emptyset = \mathcal{G} := \text{by}$ 
2604   ...
2605
2606 lemma HyperEnv.merge_assoc ( $\mathcal{G} \ \mathcal{H} \ \mathcal{J}$  : HyperEnv) :
2607    $\mathcal{G} \mid_h \mathcal{H} \mid_h \mathcal{J} = \mathcal{G} \mid_h (\mathcal{H} \mid_h \mathcal{J}) := \text{by}$ 
2608   ...
2609
2610 lemma HyperEnv.merge_comm ( $\mathcal{G} \ \mathcal{H}$  : HyperEnv) :
2611    $\mathcal{G} \mid_h \mathcal{H} \sim \mathcal{H} \mid_h \mathcal{G} := \text{by}$ 
2612   ...

```

Listing 27. Selected monoidal laws for hyperenvironment merge. Hyperenvironment merge is implemented by list append; commutativity is recovered up to deep permutation.

As with environments, hyperenvironment merge is implemented as list append. Unit and associativity are therefore inherited from append, while commutativity is stated up to deep permutation. These lemmas recover the intended exchange behaviour of hyperenvironments without replacing the list representation by a quotient or finite-set structure.

```

2621 lemma HyperEnv.Perm.refl ( $\mathcal{G}$  : HyperEnv) :
2622    $\mathcal{G} \sim \mathcal{G} := \text{by}$ 
2623   ...
2624
2625 lemma HyperEnv.Perm.symm { $\mathcal{G} \ \mathcal{H}$  : HyperEnv} :
2626    $\mathcal{G} \sim \mathcal{H} \rightarrow \mathcal{H} \sim \mathcal{G} := \text{by}$ 
2627   ...
2628
2629 lemma HyperEnv.Nodup_perm { $\mathcal{G} \ \mathcal{H}$  : HyperEnv}
2630   (hP :  $\mathcal{G} \sim \mathcal{H}$ ) :
2631   HyperEnv.Nodup  $\mathcal{G} \rightarrow$  HyperEnv.Nodup  $\mathcal{H} := \text{by}$ 
2632   ...
2633
2634 lemma HyperEnv.PairwiseDisjoint_perm { $\mathcal{G} \ \mathcal{H}$  : HyperEnv}
2635   (hP :  $\mathcal{G} \sim \mathcal{H}$ ) :
2636   HyperEnv.PairwiseDisjoint  $\mathcal{G} \rightarrow$  HyperEnv.PairwiseDisjoint  $\mathcal{H} := \text{by}$ 
2637   ...
2638
2639 lemma HyperEnv.perm_disjoint { $\mathcal{G} \ \mathcal{H} \ \mathcal{J}$  : HyperEnv}
2640   (hP :  $\mathcal{G} \sim \mathcal{H}$ ) :
2641    $\mathcal{G}.\text{disjoint } \mathcal{J} \leftrightarrow \mathcal{H}.\text{disjoint } \mathcal{J} := \text{by}$ 
2642   ...

```

Listing 28. Selected preservation lemmas for hyperenvironment permutation. These lemmas show that the structural invariants used by typing are stable under deep permutation.

The preservation lemmas in the above listing are used whenever a proof rearranges a hyperenvironment before applying a typing or transition rule. They ensure that the structural invariants of typing, such as no-duplicate names and pairwise disjointness, are stable under the deep permutation relation.

These things should be included in this appendix

% Session Types:

% - TVar

% - Types

% - Types.dual

% - Tvar.shfit / Types.shift

% - Types.subst

% - TVar.lc / Types.lc

% Processes and binding:

% - Channel

% - Proc

% - Channel.open / Proc.open

% - Channel.close / Proc.close

% - Proc.lc

% - Proc.f

% - Proc.substNames / Proc.substTypes

% - exists_one_fresh, maybe statement only

% Environments:

% - Elem

% - Env

% - Env.names

% - Env.merge

% - Env.Nodup

% - Env.disjoint

% - Env.lc / Env.shiftTypes / Env.substNames / Env.substTypes

% - selected names lemmas: names_nil, names_singleton, names_merge

% Hyperenvironments:

% - HyperEnv

% - HyperEnv.Perm

% - HyperEnv.names

% - HyperEnv.Nodup

% - HyperEnv.PairwiseDisjoint

% - HyperEnv.disjoint

% - merge laws / permutation laws

Typing:

- Full 'Typing' inductive relation, or at least all constructors in a compact listing

E Selected Transition Definitions

These things should include in this appendix

2696 Labels:

- 2697 - Mu
- 2698 - Act
- 2699 - Lbl
- 2700 - Lbl.WF
- 2701 - Lbl.f
- 2702 - Lbl.i

2703

2704 Step relations:

- 2705 - full or nearly full ProcStep_m
- 2706 - full or nearly full EnvStep_m
- 2707 - TypingStep signature
- 2708 - selected TypingStep constructors:
- 2709 one, bot, tensor, parr, par_1, par_2, syn, one_bot, tensor_parr, res, perm_env, perm_hyp

2710

2711 F Erasure Functions and Main Theorems

2712 These things should be included in this appendix

- 2713 - proc : Typing n P G -> Proc
- 2714 - env : Typing n P G -> HyperEnv
- 2715 - session_fidelity_proc
- 2716 - session_fidelity_env
- 2717 - typability_subject_reduction
- 2718 - subject_reduction

2719

2720 G Selected Proof Infrastructure

2721 These things should be included in this appendix

2722 Core typing invariants:

- 2723 - Typing_f_eq_names
- 2724 - Typing_preserves_linearity
- 2725 - maybe Typing_preserves_lc_context

2726

2727 Freshness/co-finite bridge:

- 2728 - Typing_tensor_all_fresh
- 2729 - Typing_parr_all_fresh
- 2730 - Typing_res_all_fresh if is discussed, else something similar

2731

2732 Typing inversion:

- 2733 - Typing_inv_one
- 2734 - Typing_inv_tensor
- 2735 - maybe Typing_inv_cut

2736

2737 Hyperenvironment extraction/block preservation:

- 2738 - one representative HyperEnv.Perm.extract_* statement
- 2739 - optionally one block-preservation lemma statement if it is clean enough

2740

2741

2742

2743

2744